arXiv:2611.14320v1 [cs.AI] 10 Nov 2026

Don't Just Trust Your Memory: Evidence-Linked Memory Reduces Memory-Induced Degradation in LLM Agents

Yukyeong Lee[1*] **Yumin Choi**[1*] **Jinheon Baek**[1†] **Sung Ju Hwang**[1,2†]
[1]KAIST [2]DeepAuto.ai
{yukyeong, yuminchoi, jinheon.baek, sungju.hwang}@kaist.ac.kr

Abstract

External memory enables LLM agents to improve from experience by distilling trajectories of past tasks into concise lessons for new tasks. However, memory can also lead to *memory-induced degradation*, where an agent fails on tasks it would solve without memory, often because it follows a lesson that is incorrect, or misapplies or misreads a valid one. This is because lessons are inferred from trajectories labeled only with task success or failure, yet are typically provided without the trajectories they came from, leaving the agent no way to check them. To address this, we introduce **WikiTrace**, an evidence-linked memory system designed to let agents check each lesson against its original trajectory. Specifically, motivated by the finding that a trajectory often still holds the evidence needed to catch an erroneous lesson, WikiTrace stores each lesson as a concise insight linked to a summary of why its trajectory succeeded or failed and to the raw trajectory itself. Then, on a new task, the agent starts from all insights and, whenever one lacks details needed to act, conflicts with another, or comes from a failed task, descends through its links with terminal tools to fill in missing details or correct the insight before acting. We validate WikiTrace on five agentic benchmarks, showing that it consistently outperforms existing memory approaches and a baseline given the same insights alone, while also reducing memory-induced degradation. Our code is available at https://github.com/yukyeongleee/wikitrace.

1 Introduction

Large language model (LLM) agents increasingly solve multi-step tasks by interacting with an environment (Yao et al., 2023), and external memory allows them to improve from experience without updating their parameters (Sumers et al., 2024; Hu et al., 2025b; Luo et al., 2026). Specifically, memory distills trajectories of past tasks, namely their sequences of actions and observations, into concise lessons that are reused on new tasks, allowing the agent to avoid repeated exploration and to follow procedures specific to its environment. Prior work captures such lessons in diverse forms, including verbal reflections, executable skills, instruction manuals, workflows, and reasoning strategies (Shinn et al., 2023; Zhao et al., 2024; Wang et al., 2024; Chen et al., 2024; Wang et al., 2025; Ouyang et al., 2026; Zhang et al., 2026b).

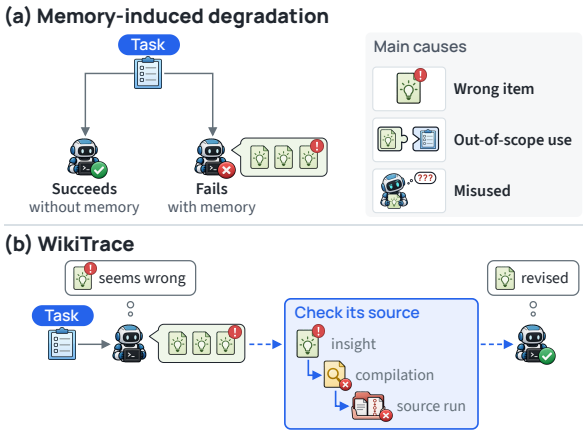

Figure 1: **Memory-induced degradation and WikiTrace.** (a) The agent solves a task without memory but fails with memory that holds a wrong insight. (b) With WikiTrace, the agent follows the wrong insight's links to its source run, revises it, and solves the task.

However, memory does not always help and can even lead to *memory-induced degradation*, where an agent fails on tasks it would solve without memory (Zhang et al., 2026a; Xiong et al., 2025; Hu et al., 2026a) (Figure 1a). Reliable experience reuse therefore requires solving more tasks while preserving those the agent already solves, but average accuracy cannot tell the two apart. We thus measure them separately, as *Gain*, the rate at which memory turns failures into successes, and *Regression*, the rate at which it turns successes into failures.

*Equal contribution; †Equal advising.

As shown in Figure 5, existing memory methods exhibit Regression comparable to, or even higher than, that of rerunning the agent without memory.

To understand this degradation, we analyze tasks that the agent consistently solves without memory but fails with it, and find that 52% of the failures trace to a memory item the agent followed (Figure 3), most often a wrong one. Wrong items arise because a trajectory comes with only a single label, success or failure, which does not reveal which of its many actions caused that outcome. Any method that learns lessons from such a trajectory therefore has to guess which actions matter and under which conditions, and each wrong guess yields a lesson that misleads the agent on new tasks. Yet, existing approaches provide lessons without the trajectories they came from, leaving the agent no way to check them, even though the evidence needed to catch an erroneous lesson often remains in that trajectory.

To this end, we present **WikiTrace**, an evidence-linked memory system that keeps each lesson together with a path back to its original trajectory (Figures 1b and 2), combining the strengths of both: a lesson concisely tells the agent what to do, whereas the trajectory faithfully records what actually happened but is too long to reread for every decision. Specifically, WikiTrace organizes its memory into three layers linked from top to bottom: each lesson is stored as a concise *insight*, which links down to a *compilation* that summarizes why its trajectory succeeded or failed, and further down to the *raw trajectory* itself. Then, on a new task, the agent starts from all insights and, whenever an insight lacks details needed to act, conflicts with another, or comes from a failed trajectory, descends through its links with terminal tools to recover missing details or revise the insight before acting.

We validate WikiTrace on five agentic benchmarks comprising eight suites, spanning simulated text worlds (Shridhar et al., 2021; Wang et al., 2022; Dagan et al., 2025) and tool-use environments (Yao et al., 2025; Barres et al., 2026; Trivedi et al., 2024). WikiTrace achieves the best performance on every suite, improving over the agent without memory by 10.7 points on average and outperforming a baseline given the same insights alone by 3.2 points, while reducing memory-induced degradation (Regression from 6.5% to 4.5%). Further analyses show that the agent consults the linked evidence more often on harder tasks, where its advantage is largest, and gains most when the evidence changes its plan. These gains hold under another backbone.

2 Related Work

2.1 Experience-Driven Procedural Memory

Language agents increasingly convert past execution experience into procedural memory that can guide future decisions. Prior methods retain verbal feedback or successful trajectories (Shinn et al., 2023; Zhao et al., 2024), accumulate executable skills (Wang et al., 2024), or distill experience into reusable guidelines, manuals, workflows, modular procedures, and multigranular scripts (Fu et al., 2024; Chen et al., 2024; Wang et al., 2025; Han et al., 2026; Fang et al., 2026). Continual approaches further update reasoning memories and playbooks with new successful and failed experience (Zhou et al., 2025; Ouyang et al., 2026; Zhang et al., 2026b; Cao et al., 2026). These approaches develop mechanisms for distilling, selecting, and updating experience for reuse. WikiTrace focuses on reassessing the resulting guidance during task execution, linking each lesson to its source evidence so that the agent can examine its basis and applicability to the current task.

2.2 Reliable Reuse of Unverified Memory

Reusing past experience does not always improve an agent's performance. Agents ignore or misinterpret relevant condensed experience (Zhao et al., 2026a), encounter retrieval competition and negative transfer during continual learning (Hu et al., 2026a), and exhibit amplified unsafe behavior when reusing execution-oriented experience collected from benign tasks (Zhao et al., 2026c). These findings motivate evaluating the failures introduced by memory alongside the tasks it helps solve. We explicitly separate Gain from Regression to highlight memory-induced degradation that aggregate accuracy obscures. Research on memory admission and revision addresses reliability by controlling which experiences and lessons are retained. PREPING validates synthetic practice trajectories before storage (Choi et al., 2026), SkillOpt admits skill updates through held-out evaluation (Yang et al., 2026), and ReMe updates or removes memories using subsequent utility (Cao et al., 2026). WikiTrace complements these approaches by focusing on how stored lessons inform decisions during task execution. This perspective treats reliable memory reuse as a question of both which lessons to retain and whether those lessons are justified in the situations where they are applied.

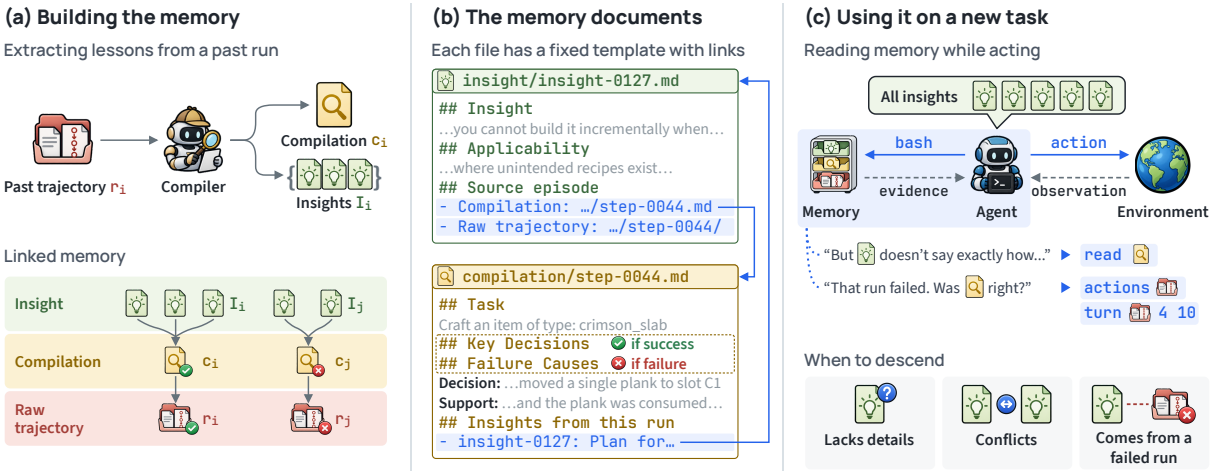

Figure 2: **Overview of WikiTrace.** (a) A compiler turns each past run into a compilation, a diagnosis of why the run succeeded or failed, and insights, each a concise lesson. All layers stay linked. (b) Each layer's documents follow a fixed template. (c) The agent starts from every insight and descends through the links under the conditions shown.

2.3 Memory Structure and Access

Agent memory design concerns both how past experience is organized and how agents access it during task execution. Hierarchical approaches organize experience at multiple levels of detail for compression, planning, and retrieval (Hu et al., 2025a; Zhang et al., 2025; Fang et al., 2026; Yang et al., 2025; Gao et al., 2025; Hu et al., 2026b), while HORMA connects filesystem summaries to raw interaction history and learns to select a minimal sufficient context (Hsu et al., 2026). Beyond memory organization, active-access methods determine when and what to retrieve during reasoning (Zhang et al., 2026c; Zhao et al., 2026b; Cai et al., 2026). Direct Corpus Interaction (DCI) provides a general approach to corpus access through terminal tools (Li et al., 2026). WikiTrace brings these two concerns together around the reassessment of procedural lessons: its structure preserves the connection between a lesson and its source execution, and its access mechanism lets the agent consult that execution when deciding how to apply the lesson.

3 WikiTrace

We first define the problem setting and examine memory-induced degradation to motivate WikiTrace. We then describe its memory construction and test-time evidence access (Figure 2).

3.1 Problem Setting

We consider an agent that reuses a fixed corpus of past runs collected without memory,

$$\mathcal{D}_{\text{train}} = \{(x_i, r_i, s_i)\}_{i=1}^{N}.$$

Each run consists of a task x_i, a trajectory r_i recording the agent's actions and observations, and a binary outcome $s_i \in \{0, 1\}$ indicating task success or failure. No gold solutions or expert explanations are available, so a run's outcome does not exactly tell us why it succeeded or failed. Thus, an insight drawn from it is not always valid.

We construct memory $\mathcal{M}$ from this corpus before any test task runs, with no additional run to validate an individual memory item. Once constructed, the memory remains fixed and read-only for an agent solving new tasks. Our objective is to improve task performance through experience reuse while limiting memory-induced degradation on tasks the agent solves without memory.

3.2 Understanding Memory-Induced Degradation

To understand how memory breaks tasks the agent can already solve, we examine tasks that the agent solves in all three repeated runs without memory but fails with memory. An LLM judge, `gpt-5.6-sol` at medium reasoning effort, compares each failed run with a no-memory run of the same task, locates the first turn where the two runs depart, and classifies the cause into one of five kinds (Figure 3; Appendix D).

In 52% of these cases, *the failure traces to a memory item the agent followed* (A to D). Most often the item itself was wrong (A, 20%). Otherwise, a valid item was applied outside its scope (B), misused by the agent (C), or followed while mem-

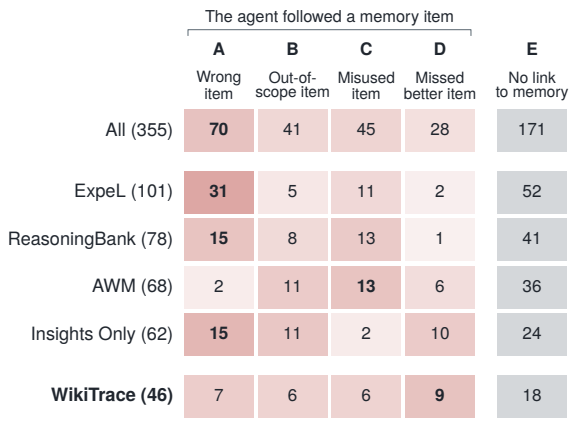

Figure 3: **Failure categories in memory-induced degradation.** LLM-judged counts of failures on tasks solved in all three trials without memory.

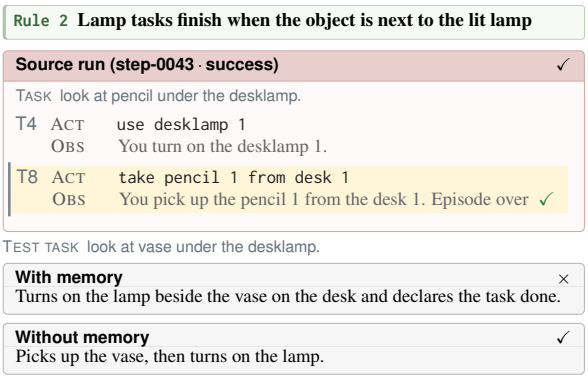

Figure 4: **A wrong lesson induces degradation.** A rule in memory says an ALFWorld lamp task ends once the object sits beside the lit lamp, which its source run contradicts. Following it, the agent fails a task it solves without memory (Figure 16).

ory held a better one (D).[1] Checking the original run a lesson was drawn from can reveal a wrong or out-of-scope item. Figure 4 shows an example: a wrong rule from ExpeL (Zhao et al., 2024) fails a task the agent solves without memory, although the run behind the rule contradicts it.

These observations motivate preserving a path from each insight back to its source run. With this path, the agent can recover the observations an insight omits and check whether the insight is supported and whether it applies to the current task before following it. WikiTrace realizes this through source-linked memory (Section 3.3) and test-time evidence access (Section 3.4).

3.3 Constructing Source-Linked Memory

WikiTrace constructs memory $\mathcal{M} = (R, C, I, E)$, consisting of raw trajectories R, compilations C, insights I, and source links E. Each run is processed independently so that every extracted lesson remains connected to the experience it came from.

Compilation and insight extraction A compiler reads a task x_i, the agent's trajectory r_i on that task, and a binary signal s_i of its success or failure, producing a compilation c_i and a set of insights I_i:

$$(c_i, I_i) = \text{Compiler}(x_i, r_i, s_i).$$

The compilation summarizes the run's key decisions and supporting observations, using separate templates for successful and failed runs (Ouyang et al., 2026; Yang et al., 2026). For successful runs, it describes decisions associated with success; for failed runs, it records plausible explanations for failure. These accounts remain interpretations of the run rather than verified explanations of its outcome. The compiler also extracts at most three insights per run. Each insight is a separate document that says what to do, when it applies, which run it came from, and whether that run succeeded. We generate insights independently for each run, so each insight has a clear source, and we never update or delete them once written. The insight pool may therefore contain redundant or conflicting insights, and deciding which one applies to the current task is left to the agent.

Source preservation and linking WikiTrace preserves each raw trajectory unchanged: the task metadata, each turn's reasoning, action and observation, and whether the run succeeded. Each insight links to both its source compilation and raw trajectory. Thus, when reassessing an insight, the test-time agent can see which run it came from and how that run was interpreted to derive it. To help the agent navigate, each layer also has an index. Appendix B provides the corpus layout, document templates, and construction algorithm.

3.4 Revisiting Source Evidence at Test Time

WikiTrace lets the agent start from the insights and descend to their source runs when an insight may not hold or does not say enough to act on now.

Initial input The agent first receives all insights, each stating what to do, when it applies, and the id of its source run. The message then lists the three past runs whose tasks are most similar to the

[1] A cause is often implicit, and an agent acts differently across runs even without memory. Many cases therefore show no link to any memory item (E).

Table 1: **Main results.** Mean over three trials, ± the standard deviation across trials in small type. Avg. averages the eight suites. Best results are bolded and second-best results are underlined.

Method	ALFWorld		ScienceWorld	PlanCraft		τ^3-bench Retail	AppWorld		Avg.
	Seen	Unseen		Seen	Unseen		Normal	Challenge	
No Memory	94.4±2.0	94.3±1.9	83.7±0.8	72.3±1.9	56.3±1.2	86.8±1.9	75.4±3.4	72.7±2.2	79.5±1.6
AWM	94.7±1.2	97.3±0.9	88.9±4.7	76.3±3.7	57.3±2.1	85.5±2.1	79.3±0.7	73.4±1.7	81.6±0.3
ReasoningBank	92.1±2.4	92.8±0.7	85.0±1.3	84.7±2.6	61.3±3.4	86.4±1.6	80.8±1.5	70.7±3.0	81.7±0.5
ExpeL	94.4±0.9	98.0±0.4	91.9±1.6	74.0±8.5	51.0±3.6	82.5±5.1	80.5±5.7	70.0±3.7	80.3±1.6
Insights Only	96.7±0.5	97.3±1.3	94.9±0.6	88.7±6.1	73.0±10.7	85.1±2.7	86.8±0.4	73.7±1.4	87.0±2.5
WikiTrace (ours)	**98.7**±0.5	**98.8**±0.7	**96.1**±1.4	**93.3**±3.4	**81.3**±8.5	**88.2**±3.9	**89.5**±1.5	**75.8**±2.2	**90.2**±1.6

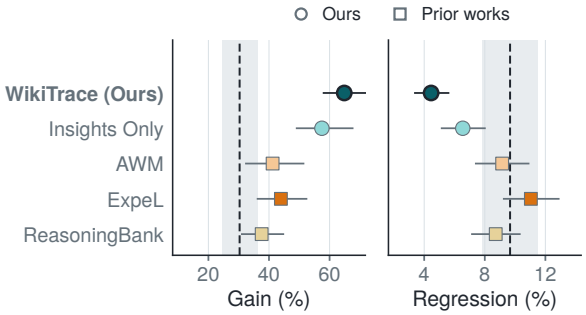

Figure 5: **Gain and Regression of WikiTrace and memory-based baselines**, averaged over eight suites with 95% intervals.

current one, with the ids and titles of the insights drawn from each. These runs give the agent starting points for its search.

Descent to source runs To let the agent examine an insight before following it, the prompt asks it to *descend* from the insight to its source run when the insight lacks the details needed to act on it, comes from a failed run, or disagrees with another insight. It asks for another descent whenever an action fails. The source-linked memory is mounted as read-only files, and the prompt encourages the agent to search them with `read` and `bash` for the fact that settles the question, such as a command format. Figure 27 in Appendix F gives the full prompt.

4 Experiments

4.1 Experimental Setup

Benchmarks We evaluate on eight suites across five benchmarks. Three are simulated text worlds, where the agent issues one text command at a time: ALFWorld (Shridhar et al., 2021) (household tasks), ScienceWorld (Wang et al., 2022) (science experiments) and PlanCraft (Dagan et al., 2025) (Minecraft crafting). Two are tool-use environments: AppWorld (Trivedi et al., 2024) poses everyday tasks through application APIs, and τ^3-bench Retail (Yao et al., 2025; Barres et al., 2026) combines conversation with a simulated customer and tool calls over an order database. AppWorld includes Test-Normal and Test-Challenge; the latter requires unseen applications. Appendix A.1 details suite construction and splits.

Baselines **No Memory** is the same agent with nothing added to its prompt or files. **AWM** (Wang et al., 2025) abstracts successful trajectories into reusable workflows. **ExpeL** (Zhao et al., 2024) keeps a list of free-form rules and reviews the whole list against each successful trajectory to add, edit or remove rules. **ReasoningBank** (Ouyang et al., 2026) distills successful and failed trajectories into reasoning strategies and retrieves them by task similarity. We adapt their official implementations to our offline protocol (Appendix A.3). **Insights Only** gives the same insights as WikiTrace but mounts no source compilations or raw trajectories and omits the prompt to descend, so the two differ only in access to source runs.

Protocol We run three trials per suite and method. In each trial, every method builds its memory once from the same No Memory runs of the training tasks and is tested on the same test tasks. No method may make additional runs to validate its memory. All methods use the same agent: a Pi terminal agent (Earendil Works, 2025) given each benchmark's ReAct instruction. The agent and memory builders use `deepseek-v4-flash`[2] with high reasoning effort. We report each benchmark's original metric, averaged over the three trials. See Appendix A.2 for implementation details.

[2]The 2026-04-23 release.

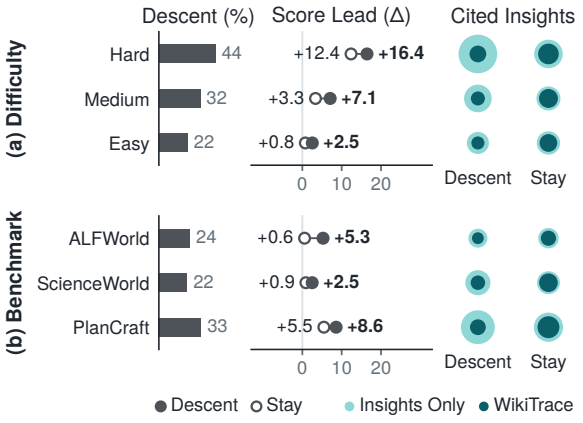

Figure 6: **WikiTrace against Insights Only by descent** in the text-world benchmarks. Score Lead Δ is WikiTrace minus Insights Only, and circle radius gives the insights reported as used. Difficulty: No Memory solves the task in none (Hard), one or two (Medium) or all three (Easy) trials.

4.2 Main Results

Table 1 shows that WikiTrace achieves the highest performance on all eight suites, averaging 90.2 compared with 79.5 for No Memory. It also exceeds Insights Only by 3.2 points on average, demonstrating the benefit of source evidence and the descent prompt beyond the same insights alone. Notably, WikiTrace is the only memory method that matches or exceeds No Memory on every suite, suggesting that revisiting source evidence helps mitigate memory-induced degradation.

4.3 Gain and Regression

Aggregate performance does not distinguish improvements on previously failed tasks from failures on previously solved tasks. We therefore report Gain and Regression separately.

Gain and Regression. For task i, let B_i denote the majority No Memory outcome across three trials and $M_{i,t}$ the outcome with memory in trial t. Gain measures success on tasks No Memory fails, while Regression measures failure on tasks No Memory solves:

$$\text{Gain} = \Pr_{i,t}(M_{i,t} = 1 \mid B_i = 0),$$
$$\text{Regression} = \Pr_{i,t}(M_{i,t} = 0 \mid B_i = 1).$$

Figure 5 reports both rates averaged across suites, with 95% bootstrap confidence intervals over tasks (Appendix C.1). To account for variation across runs, we evaluate three additional No Memory trials against the same reference outcomes B_i. The dashed line shows this rerun baseline, measuring outcome changes without memory.

WikiTrace achieves a Regression of 4.5%, compared with 6.5% for Insights Only and 9.7% for the rerun baseline. Its Regression is lower than Insights Only on seven of the eight suites, and it is the only method whose Regression confidence interval lies entirely below the rerun band. This reduction accompanies higher Gain: WikiTrace exceeds the rerun baseline by 35 percentage points, compared with 27 for Insights Only and 7 to 14 for existing memory methods. Because WikiTrace and Insights Only differ only in access to source runs, the higher Gain and lower Regression are attributable to that access. Relative to Insights Only, wrong or out-of-scope items cause half as many of WikiTrace's regressions (Figure 3). Appendix C provides results for individual suites.

5 Analysis

5.1 When Does Descent Help?

Insights Only gives the agent the same insights, so comparing it with WikiTrace on the same task and trial shows what access to source runs adds. We split WikiTrace's test runs in the three text-world benchmarks into Descent runs, which open a compilation or raw trajectory, and Stay runs, which do not (Figure 6). WikiTrace descends more often on harder tasks, in 22% of runs on Easy tasks and 44% on Hard ones. Descent runs lead Insights Only by 1.6 to 4.7 points more than Stay runs at every difficulty and on every benchmark. The gap between Descent and Stay grows with difficulty, from 1.8 points on Easy tasks to 3.7 on Medium and 4.1 on Hard ones. These results suggest that source runs help more on harder tasks. The agent chooses when to descend, however, so Descent and Stay runs differ in more than the descent.

Stay runs also lead Insights Only, by 0.6 to 12.4 points. In Stay runs, WikiTrace reports 19 to 28% fewer insights as used,[3] which suggests more selective use of memory. The reduction is larger in Descent runs, 38 to 57%.

5.2 Why and How Does WikiTrace Descend?

Section 5.1 shows when descent helps, but not why the agent descends or what the opened run changes. To find out, an LLM judge reads the agent's reasoning around each descent in the three text-world benchmarks and labels three things: its *trigger*,

[3] After each run, the agent names the insights it applied.

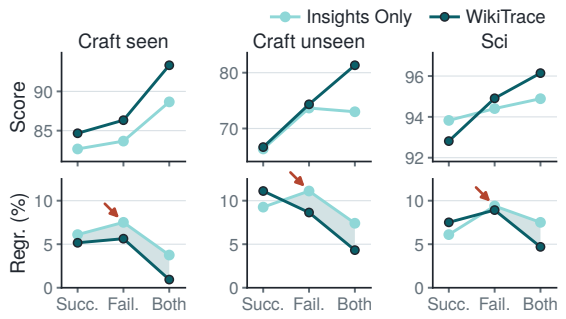

Figure 7: **Score and Regression by memory subset.** Memory is built from successful runs (Succ.), failed runs (Fail.) or both. Arrows mark Insights Only's higher Regression with failed runs.

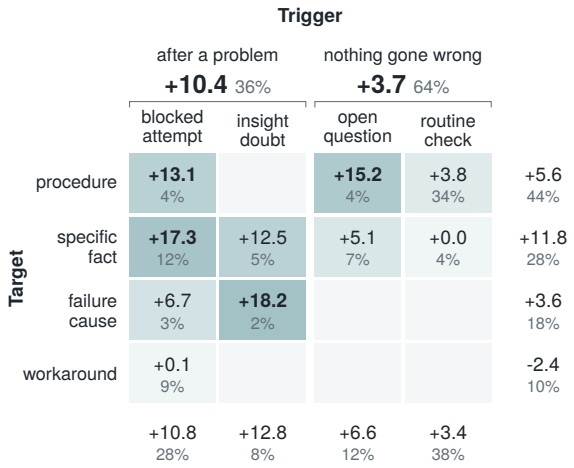

Figure 8: **Why WikiTrace descends and what it seeks.** WikiTrace's score lead over Insights Only by trigger and target, with descent shares in gray.

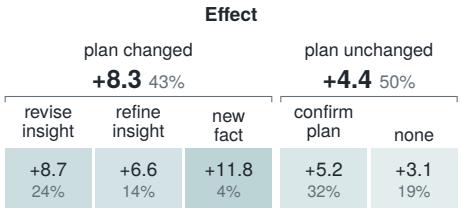

Figure 9: **How WikiTrace uses evidence.** WikiTrace's score lead over Insights Only by effect, with descent shares in gray.

what prompted the descent; its *target*, what the agent looked for in the opened run; and its *effect*, what the run contributed to the agent's next actions (Figures 8 and 9). The judge, `gpt-5.6-sol` at medium reasoning effort, agrees with a second judge, `GLM-5.3`, at Cohen's κ of 0.77 on trigger and 0.63 on target. Effect is harder to judge, so its label is the majority of three judges (Appendix E.1).

Why it descends and what it seeks WikiTrace's advantage over Insights Only concentrates on descents made after a problem. Descents that follow a blocked attempt, an observation that blocked the agent's plan, or a doubt about an insight make up 36% of descents and lead Insights Only by 10.4 points, against 3.7 points for descents made with nothing gone wrong (Figure 8). The largest advantage, 17.3 points, comes when a blocked attempt sends the agent after a specific fact in past runs, which Insights Only has no way to look up. Routine checks are the most common descent (38%), and 90% of them read a similar run's procedure, with a small advantage of 3.4 points.

How it uses source evidence The opened run changes the agent's plan in 43% of descending runs, by revising an insight (24%), adding a detail an insight leaves out (14%), or supplying a fact no insight covers (4%). These runs lead Insights Only by 8.3 points, against 4.4 points when the opened run only confirms the plan or goes unused (Figure 9). Figure 11 illustrates such a revision, with additional cases in Appendix E.2.

5.3 Can Agents Reliably Learn from Failed Experiences?

Failed runs contain useful experience, but the lessons drawn from them depend on how the failure is interpreted. To examine their value for memory, we compare WikiTrace and Insights Only using insights from successful runs, failed runs, or both on PlanCraft and ScienceWorld. Figure 7 shows that both methods perform better with failed runs alone than with successful runs alone, and perform best with both. Failed runs therefore add guidance beyond what successful runs give.

However, this benefit accompanies a greater risk of regression when the extracted insights are provided alone. Insights Only has its highest Regression with failed runs alone, reaching 9.3% on PlanCraft and 9.4% on ScienceWorld. With the same insights, WikiTrace reduces these rates to 7.1% and 8.9%. These findings support keeping failed runs with their source evidence, so that agents can revisit the basis of a lesson before reusing it.

5.4 How Cost-Effective Is WikiTrace?

To examine whether WikiTrace's advantage comes from greater memory construction cost, we increase existing methods' construction budgets by running their builders over the same training trajectories nine times for AWM and three times for ExpeL and ReasoningBank. Figure 10 shows that WikiTrace retains the highest average performance

Method	Craft		Sci	Avg.
	Seen	Unseen		
NM	68.7±3.3	55.3±2.1	89.6±1.5	71.2±0.3
AWM	66.0±2.8	47.0±2.9	91.7±1.0	68.2±0.5
ReasoningBank	74.7±4.2	53.3±5.2	90.9±1.9	73.0±2.3
ExpeL	79.0±12.8	67.0±14.1	88.6±1.4	78.2±8.5
Insights Only	92.3±2.1	87.7±3.3	92.6±0.8	90.9±1.6
WT	**95.7**±1.7	**89.0**±1.6	**93.3**±0.8	**92.7**±0.4

Table 2: **Results with `GLM-5.3-flash`**. Mean over three trials. NM is No Memory and WT is WikiTrace.

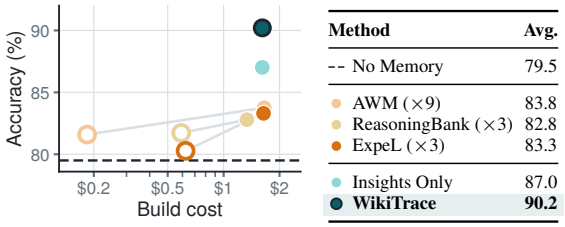

Figure 10: **Accuracy against build cost**, averaged over the eight suites. A hollow mark is a default build and a filled mark the build run N times. The dashed line shows No Memory.

even with these larger baseline budgets. At test time, WikiTrace costs 1.17 times as much per task as Insights Only, averaged across suites, while improving average performance by 3.2 points. These results suggest that WikiTrace improves performance with modest additional test-time cost.

5.5 Does WikiTrace Generalize?

WikiTrace also achieves the highest performance on all three evaluated suites with `GLM-5.3-flash` (Table 2). We evaluate all methods on PlanCraft and ScienceWorld at medium reasoning effort, using training executions generated by the same model to construct memory. WikiTrace averages 92.7, exceeding Insights Only by 1.8 percentage points and No Memory by 21.5. Each existing memory method falls below No Memory on at least one suite. This pattern mirrors the DeepSeek results, supporting the benefit of source-linked memory across the two models.

5.6 Resolving Conflicting Insights

Figure 11 illustrates how WikiTrace resolves conflicting insights by revisiting source evidence. In PlanCraft, an insight from a failed run incorrectly rules out a valid procedure that another recommends. The successful run's observations show that the procedure works. WikiTrace revises the insight and succeeds, whereas Insights Only, given identical insights, follows the wrong guidance and fails. This case shows how raw trajectories enable agents to reassess interpretations embedded in insights and compilations.

6 Conclusion

Lessons extracted from past executions are not validated by their outcomes alone, and reusing them without scrutiny can cause memory-induced degradation. We introduced WikiTrace, which links insights to source compilations and raw trajectories so that agents can recover missing details and reassess their guidance. Across five benchmarks, WikiTrace achieves the highest performance on every suite while increasing Gain and reducing Regression compared with injecting the same insights alone. These findings suggest that agent memory should preserve not only reusable lessons, but also the evidence needed to reassess them.

Limitations

Memory is built from binary trajectory outcomes, so a compilation's diagnosis of a failure, and any insight drawn from it, may be wrong. A source link lets the agent check an insight against its run but does not verify the insight. WikiTrace places every insight in the first message so that the agent never misses one, and the prompt therefore grows with the memory. A memory too large to place in full would need the agent to find the insights it needs, a retrieval problem this work leaves open. The causes of degradation (Section 3.2) and the descent labels (Section 5.2) come from an LLM judge that reads the agent's reasoning, so they describe what that reasoning states, not what produced the outcome.

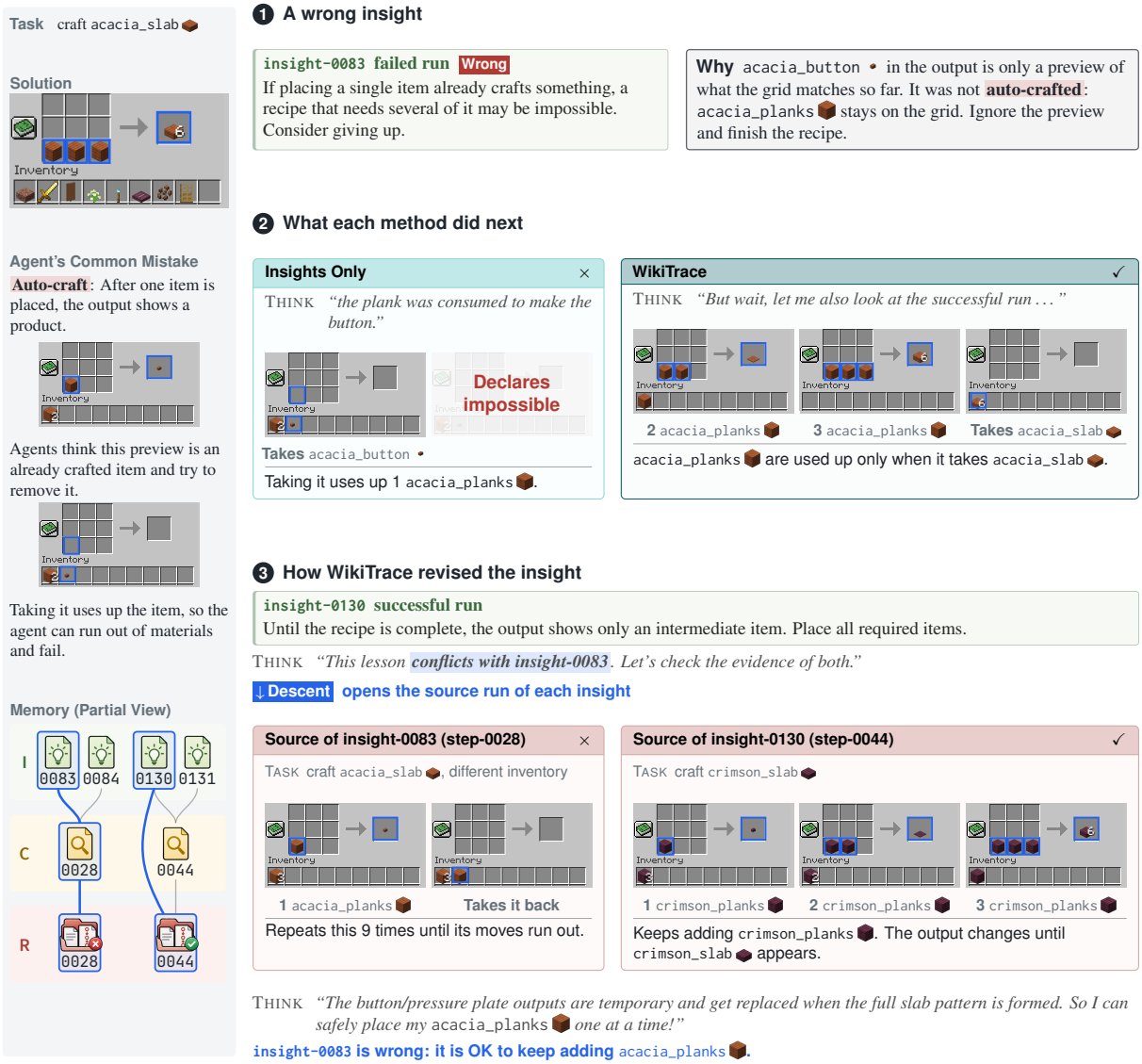

Figure 11: **Qualitative example: revising a wrong insight (PlanCraft).** Insight-0083 leads agents into the auto-craft mistake. Insights Only follows it and fails, but WikiTrace revises it through descent and succeeds.

References

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2026. τ^2-bench: Evaluating conversational agents in a dual-control environment. In *Forty-third International Conference on Machine Learning*.

Yuxuan Cai, Wei Li, Jie Zhou, Qin Chen, Xin Li, Bo Zhang, and Liang He. 2026. Ask only when needed: Proactive retrieval from memory and skills for experience-driven lifelong agents. *arXiv preprint arXiv:2604.20572*.

Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. 2026. Remember me, refine me: A dynamic procedural memory framework for experience-driven agent evolution. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. 2024. AutoManual: Constructing instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems*.

Yumin Choi, Sangwoo Park, Minki Kang, Jinheon Baek, and Sung Ju Hwang. 2026. PREPING: Building agent memory without tasks. *arXiv preprint arXiv:2605.13880*.

Gautier Dagan, Frank Keller, and Alex Lascarides. 2025. Plancraft: an evaluation dataset for planning with LLM agents. In *Second Conference on Language Modeling*.

Earendil Works. 2025. Pi Agent Harness. `https://github.com/earendil-works/pi`. Official Website: `https://pi.dev/`.

Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2026. Memp: Exploring agent procedural memory. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. 2024. AutoGuide: Automated generation and selection of context-aware guidelines for large language model agents. In *Advances in Neural Information Processing Systems*.

Pengyu Gao, Jinming Zhao, Xinyue Chen, and Long Yilin. 2025. An efficient context-dependent memory framework for LLM-centric agents. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*.

Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang, Victor Rühle, and Saravan Rajmohan. 2026. LEGOMem: Modular procedural memory for multi-agent LLM systems for workflow automation. In *Proceedings of the 2026 International Conference on Autonomous Agents and Multiagent Systems*.

Hao-Lun Hsu, Nikki Lijing Kuang, Boyi Liu, Zhewei Yao, and Yuxiong He. 2026. Organize then retrieve: Hierarchical memory navigation for efficient agents. *arXiv preprint arXiv:2606.11680*.

Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. 2025a. HiAgent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Qisheng Hu, Quanyu Long, and Wenya Wang. 2026a. When continual learning moves to memory: A study of experience reuse in LLM agents. *arXiv preprint arXiv:2604.27003*.

Yuyang Hu, Jiongnan Liu, Jiejun Tan, Yutao Zhu, and Zhicheng Dou. 2026b. Memory matters more: Event-centric memory as a logic map for agent searching and reasoning. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, and 28 others. 2025b. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*.

Zhuofeng Li, Haoxiang Zhang, Cong Wei, Pan Lu, Ping Nie, Yi Lu, Yuyang Bai, Shangbin Feng, Hangxiao Zhu, Ming Zhong, Yuyu Zhang, Jianwen Xie, Yejin Choi, James Zou, Jiawei Han, Wenhu Chen, Jimmy Lin, Dongfu Jiang, and Yu Zhang. 2026. Beyond semantic similarity: Rethinking retrieval for agentic search via direct corpus interaction. *arXiv preprint arXiv:2605.05242*.

Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren. 2023. SwiftSage: A generative agent with fast and slow thinking for complex interactive tasks. In *Advances in Neural Information Processing Systems*, volume 36.

Jinghao Luo, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang, and Jing Ma. 2026. From storage to experience: A survey on the evolution of LLM agent memory mechanisms. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2024. Cognitive architectures for language agents. *Transactions on Machine Learning Research*.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.

Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. 2022. ScienceWorld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, Proceedings of Machine Learning Research.

Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. 2025. How memory management impacts LLM agents: An empirical study of experience-following behavior. *arXiv preprint arXiv:2505.16067*.

Wei Yang, Jinwei Xiao, Hongming Zhang, Qingyang Zhang, Yanna Wang, and Bo Xu. 2025. Coarse-to-fine grounded memory for LLM agent planning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. 2026. SkillOpt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

Dylan Zhang, Yanshan Lin, Zhengkun Wu, Yihang Sun, Bingxuan Li, Dianqi Li, and Hao Peng. 2026a. Useful memories become faulty when continuously updated by LLMs. *arXiv preprint arXiv:2605.12978*.

Guibin Zhang, Muxin Fu, Kun Wang, Frank Wan, Miao Yu, and Shuicheng Yan. 2025. G-Memory: Tracing hierarchical memory for multi-agent systems. In *Advances in Neural Information Processing Systems*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Wenyuan Zhang, Xinghua Zhang, Haiyang Yu, Shuaiyi Nie, Bingli Wu, Juwei Yue, Tingwen Liu, and Yongbin Li. 2026c. ExpSeek: Self-triggered experience seeking for web agents. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*.

Weixiang Zhao, Yingshuo Wang, Yichen Zhang, Yang Deng, Yanyan Zhao, Wanxiang Che, Bing Qin, and Ting Liu. 2026a. Large language model agents are not always faithful self-evolvers. In *Proceedings of the 43rd International Conference on Machine Learning*, Proceedings of Machine Learning Research.

Weixiang Zhao, Yingshuo Wang, Yichen Zhang, Yanyan Zhao, Yu Zhang, Yang Wu, Dandan Tu, Bing Qin, and Ting Liu. 2026b. Rethinking experience utilization in self-evolving language model agents. *arXiv preprint arXiv:2605.07164*.

Weixiang Zhao, Yichen Zhang, Yingshuo Wang, Yang Deng, Yanyan Zhao, Xuda Zhi, Yongbo Huang, Hao He, Wanxiang Che, Bing Qin, and Ting Liu. 2026c. On safety risks in experience-driven self-evolving agents. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. 2025. Memento: Fine-tuning LLM agents without fine-tuning LLMs. *arXiv preprint arXiv:2508.16153*.

Appendix

Table 3: **Benchmark suites.** One training set per benchmark serves all of its test suites.

Benchmark	Suite	Train	Test	What the test holds out	Metric
ALFWorld	`valid-seen`	50	101	Games in training rooms.	Success rate
	`valid-unseen`	50	134	Games in rooms absent from training.	Success rate
ScienceWorld	`scienceworld`	50	100	Descriptions absent from training, same task types.	Mean score
PlanCraft	`plancraft-seen`	50	100	Examples with training target items.	Success rate
	`plancraft-unseen`	50	100	Examples with target items absent from training.	Success rate
AppWorld	`test-normal`	54	111	Tasks over training apps.	TGC
	`test-challenge`	54	99	Tasks needing apps absent from training.	TGC
τ^3-bench	`retail`	38	76	New requests, mostly from training customers.	Pass rate

A Experimental Details

A.1 Benchmark Suite Construction

Table 3 lists the suites. Every draw is seeded.

ALFWorld Training holds 50 games of the training split, eight or nine per task type, in rooms that also appear in valid_seen. `valid-seen` is the full set of 101 valid_seen games filtered to rooms visited by the 50 training games, and `valid-unseen` uses all 134 valid_unseen games, set in four rooms absent from training.

ScienceWorld We use ten of the 30 task types, boil, melt, freeze, change-the-state-of-matter-of, use-thermometer, measure-melting-point-known-substance, measure-melting-point-unknown-substance, test-conductivity, test-conductivity-of-unknown-substances, and chemistry-mix. Each meets three conditions.

- Its variations pose distinct tasks.
- Its training and test descriptions are largely disjoint.
- Its reference solutions fit the simulator's 100-step limit.

Training holds five variations per type from the training split, and test holds 100 variations from the test split whose description does not occur in training.

PlanCraft We use solvable examples only, weighted toward medium and hard ones. Training holds 50 examples of the training split with distinct target items that also occur in the test split. `plancraft-seen` holds 100 test examples whose target is a training target, and `plancraft-unseen` 100 whose target occurs in no solvable training example.

AppWorld A scenario groups three tasks and is solved only when all three pass, so every suite keeps scenarios whole. We use difficulty 2 and 3 only. Training holds all 18 such training scenarios (54 tasks), `test-normal` all 37 of the Test-Normal split (111 tasks), and `test-challenge` a draw of 33 of the 65 difficulty-3 scenarios of the Test-Challenge split (99 tasks). TGC counts tasks and SGC scenarios.

τ^3-bench (Retail) We set aside the authors' splits and divide the 114 tasks into 38 training and 76 test tasks, grouped by customer, so every customer with several tasks appears on both sides. The simulated user is `gpt-4.1`, and the judge of conversation assertions is `gpt-5.4-mini` at medium reasoning effort.

A.2 Agent Harness

Table 4 lists each benchmark's harness. An episode ends when the task is done, the step budget runs out, or the agent replies without a tool call. The Pi turn cap is only a guard, since a turn may hold memory reads that are not steps.

Tool `act(command)` Sends one text command to the game and returns its observation. We add a *done* command that ends the episode, so the agent can stop early.

Tool `take_action(action, argument)` Joins the action and its argument with a colon for the benchmark's own parser. PlanCraft's actions are *move* and *smelt* (source slot, target slot, quantity), *search* (a recipe, seeded), and *impossible*, which ends the episode. We drop the authors' *think*, since the agent has its own reasoning channel. In τ^3-bench an action is one of the 16 retail tools, or

Table 4: **Agent harness per benchmark.** The instruction, exemplars, and step budget follow the listed ReAct baseline. A step is one environment tool call, except in τ^3-bench, which counts agent, user, and tool turns as its authors do.

Benchmark	ReAct baseline	Tool	Max steps	Max Pi turns
ALFWorld	ReAct loop (Yao et al., 2023)	`act`	49	150
ScienceWorld	SwiftSage's ReAct baseline (Lin et al., 2023)	`act`	100	400
PlanCraft	The authors' prompt (Dagan et al., 2025)	`take_action`	30	150
AppWorld	AppWorld's ReAct code agent (Trivedi et al., 2024)	`exec`	50	150
τ^3-bench	τ^2-bench's LLM agent (Barres et al., 2026)	`take_action`	200	150

Table 5: **Compared methods.** Formatted Rules are rules with required fields. WikiTrace links each to the compilation and raw trajectory it came from. Access is inject (I), retrieve (R) or traverse (T).

	Learns from			
Method	**Succ.**	**Fail.**	**Memory**	**Access**
AWM	✓	–	Procedures	I
ExpeL	✓	–	Rules	I
ReasoningBank	✓	✓	Formatted Rules	R
Insights Only	✓	✓	Formatted Rules	I
WikiTrace (ours)	✓	✓	+ Linked Evidence	I + T

respond with a message to the simulated user. An episode also ends on the user's stop, transfer, or out-of-scope token, or after 10 tool errors.

Tool `exec(code)` Runs Python in the task's shell, which holds the benchmark's API bindings and keeps state across calls, and returns the output.

A.3 Compared Methods: Details

We run the official implementations of the existing memory methods with their own induction prompts and retrieval (Wang et al., 2025; Zhao et al., 2024; Ouyang et al., 2026), changed only to fit our protocol (Section 4.1).

- **Offline.** AWM runs in its offline mode, which induces workflows once from training examples. ReasoningBank updates memory after every test task, and we instead build it once. Every method thus builds memory once from the shared no-memory training trajectories and freezes it.
- **True outcome.** Each builder receives the binary outcome, in place of an LLM judge (AWM, ReasoningBank) or the environment reward (ExpeL).
- **Single pass.** Each training trajectory is read once. ExpeL thus has no failed and successful attempts at one task to compare, which its Reflexion (Shinn et al., 2023) retries would provide, so it draws insights from successes alone, and we omit its exemplar retrieval.

Algorithm 1 WIKITRACE: Memory Construction

Require: Fixed corpus $\mathcal{D}_{\text{train}} = \{(x_i, r_i, s_i)\}_{i=1}^{N}$
Ensure: Frozen evidence-linked memory corpus $\mathcal{M}$
$R \leftarrow \{r_i\}_{i=1}^{N}$, $C \leftarrow \varnothing$, $I \leftarrow \varnothing$, $E \leftarrow \varnothing$
for $i = 1, \ldots, N$ **do**
  $(c_i, I_i) \leftarrow \textsc{Compiler}(x_i, r_i, s_i)$
  $C \leftarrow C \cup \{c_i\}$, $I \leftarrow I \cup I_i$
  $E \leftarrow E \cup \{(u, i) \mid u \in I_i\}$
end for
Build $\textsc{Idx}(L)$ for $L \in \{R, C, I\}$ ▷ document titles
$\mathcal{M} \leftarrow \textsc{Freeze}(R, C, I, E)$ ▷ read-only
return $\mathcal{M}$

B Memory System Details

B.1 Corpus and Memory Layout

WikiTrace mounts its corpus read-only at `/work/corpus`. Each index is generated by code from the layer it lists, and the corpus has no root index.

WikiTrace corpus layout

```
/work/corpus/
|-- corpus.json                build stamp
|-- raw/
|   |-- _index.md              every run and its outcome
|   `-- step-NNNN/
|       |-- metadata.md        task, outcome, step count
|       `-- solver.full.jsonl  the agent's turn log
|-- compilation/
|   |-- _index.md              every compilation by title
|   `-- step-NNNN.md           one per run
`-- insight/
    |-- _index.md              insight titles by task type
    `-- insight-NNNN.md        one per candidate insight
```

A raw run keeps the record the agent left, unchanged. For one ALFWorld training run, which succeeded in 20 `act` calls, the box below shows its `metadata.md` and Figure 12 an excerpt of its `solver.full.jsonl`. The turn log holds the system and user messages, one line per environment call with the model's reasoning and the observation, and a closing usage record.

raw/step-0000/metadata.md

```
# alfworld_0000_pick_and_place_simple-Kettle-None-Shelf-1

- task_type: pick_and_place_simple
- instruction: Your task is to: put a kettle in shelf.
- initial_observation: You are in the middle of a room. Looking
 quickly around you, you see a cabinet 9, a cabinet 8, a
cabinet 7, a cabinet 6, a cabinet 5, a cabinet 4, a cabinet 3,
a cabinet 2, a cabinet 1, a coffeemachine 1, a countertop 3, a
countertop 2, a countertop 1, a drawer 9, a drawer 8, a drawer
7, a drawer 6, a drawer 5, a drawer 4, a drawer 3, a drawer 2,
a drawer 1, a fridge 1, a garbagecan 1, a microwave 1, a shelf
3, a shelf 2, a shelf 1, a sinkbasin 1, a stoveburner 4, a
stoveburner 3, a stoveburner 2, a stoveburner 1, and a toaster
1.
- success: True
- steps: 20 act calls
```

B.2 Evidence-Linked Document Formats

The compiler's structured output is rendered as markdown with a frontmatter block. In the mounted corpus, a compilation lists links to the insight documents made from its candidate insights in place of the insights themselves.

Success compilation

```
---
type: trajectory-compilation
title: [success] <title of the case>
description: <one-line description>
step_id: step-NNNN
outcome: success
---

# Compilation (success)

## Task
<the run's task fields>

## Key Decisions
### Decision 1
**Decision**: <what the agent chose to do>
**Support**: <what showed it was necessary>

## Insights from this run
- insight-NNNN: <title> (/work/corpus/insight/insight-NNNN.md)
```

Failure compilation

```
---
type: trajectory-compilation
title: [failure] <title of the case>
description: <one-line description>
step_id: step-NNNN
outcome: failure
---

# Compilation (failure)

## Task
<the run's task fields>

## Failure Causes
### Cause 1
**Decision**: <what the agent did, or did not do>
**Support**: <what showed it caused the failure>

## Insights from this run
- insight-NNNN: <title> (/work/corpus/insight/insight-NNNN.md)
```

raw/step-0000/solver.full.jsonl, excerpt


```
{"type": "message", "role": "system", "content": "Interact with a household to solve a task. Here ...", "turn": 0}
{"type": "message", "role": "user", "content": "Here is the task.\nYou are in the middle of a roo ...", "turn": 1}
{"turn": 2, "type": "function_call", "role": "assistant", "name": "act", "arguments": {"command": "go to cabinet 1"}, "reasoning": "Let me
start by exploring the room to find a kettle. I'll ch ...", "result": "You arrive at cabinet 1. The cabinet 1 is closed.", "is_error":
false}
{"turn": 3, "type": "function_call", "role": "assistant", "name": "act", "arguments": {"command": "open cabinet 1"}, "reasoning": "Let me
open cabinet 1 to see what's inside.", "result": "You open the cabinet 1. The cabinet 1 is open. In it, you se ...", "is_error": false}
<turns 4 to 20: seventeen more act calls of the same form>
{"turn": 21, "type": "function_call", "role": "assistant", "name": "act", "arguments": {"command": "move kettle 1 to shelf 1"}, "reasoning":
 "I can put the kettle on shelf 1.", "result": "You move the kettle 1 to the shelf 1.\n\nEpisode over", "is_error": false}
{"turn": 22, "type": "episode_over"}
{"type": "message", "role": "assistant", "content": "", "reasoning": "", "turn": 24}
{"type": "answer", "role": "assistant", "model_requested": null, "model_returned": "deepseek/deepseek-v4-flash", "parsed": null, "usage":
{"prompt_tokens": 52444, "completion_tokens": 1172, "...": "..."}, "read_set": [], "turn": 25}
```


Figure 12: **Excerpt of `raw/step-0000/solver.full.jsonl`, the turn log of one ALFWorld training run.** Long strings are cut short.

Insight document

```
---
type: insight
id: insight-NNNN
title: <insight title>
task_type: <task type of the source run>
source_step: step-NNNN
---

## Insight
<what to do>

## Applicability
<when it applies, and when it does not>

## Source episode
step-NNNN (<success|failure>)
- Task: <the source run's instruction>
- Compilation: /work/corpus/compilation/step-NNNN.md
- Raw trajectory: /work/corpus/raw/step-NNNN/
```

On AppWorld an insight document carries `apps`, the apps its source run called, in place of `task_type`.

B.3 Construction Algorithm

WikiTrace builds its memory in one pass over a fixed corpus of no-memory trajectories, with one compiler call per trajectory (Algorithm 1). Each candidate insight becomes its own document, tagged with its source run and that run's outcome and task type. Insights are neither merged nor filtered, so $|I| = \sum_i |I_i|$.

C Gain and Regression with Confidence Intervals

C.1 Confidence Intervals

Table 6 counts the tasks that each method moved per benchmark, the numerators and denominators behind the rates. Figure 13 draws the rates of Section 4.3 for each suite with 95% confidence intervals and the No Memory floor.

Intervals. We resample tasks with replacement within each suite 10,000 times. A resampled task keeps its three No Memory trials and its three trials with each method, and every rate is recomputed on the same resample, so the methods stay paired. The suite average is taken within each resample, and its interval spans the 2.5th to 97.5th percentiles of these averages.[4]

No Memory floor. B_i is the majority of three No Memory trials, so a fresh run without memory can disagree with it by chance. We ran three further No Memory trials and score them against the same B_i exactly as a method: the Regression floor is the share of their trials that fail a task with $B_i = 1$, and the Gain floor the share that solve a task with $B_i = 0$. Its interval comes from the same task resamples, in which each task keeps all six No Memory trials. A method's excess is its rate minus the floor, with the interval of that difference over the resamples. A positive excess is added Gain, or, for Regression, added harm.

[4] Resamples with no eligible ALFWorld-seen task for Gain (4.8%) are dropped.

Table 6: **Gain and Regression counts per benchmark.** Tasks moved on one trial, averaged over trials and summed over suites, with the rate over the eligible tasks (first row) in small type. Bold is best and underlined second best.

	ALFWorld		ScienceWorld		PlanCraft		τ^3-bench		AppWorld	
Method	Gain ↑	Reg. ↓	Gain ↑	Reg. ↓	Gain ↑	Reg. ↓	Gain ↑	Reg. ↓	Gain ↑	Reg. ↓
Eligible tasks	10	225	29	71	75	125	6	70	52	158
AWM	7.0 (70%)	6.0 (3%)	13.3 (46%)	12.3 (17%)	21.7 (29%)	13.0 (10%)	2.3 (39%)	7.3 (10%)	17.0 (33%)	14.3 (9%)
ReasoningBank	4.7 (47%)	12.3 (5%)	11.0 (38%)	9.0 (13%)	30.3 (40%)	9.3 (7%)	2.0 (33%)	6.3 (9%)	18.0 (35%)	16.3 (10%)
ExpeL	8.0 (80%)	6.3 (3%)	14.7 (51%)	11.7 (16%)	16.0 (21%)	16.0 (13%)	2.0 (33%)	9.3 (13%)	21.0 (40%)	20.3 (13%)
Insights Only	8.0 (80%)	5.0 (2%)	18.7 (64%)	5.3 (8%)	43.3 (58%)	6.7 (5%)	**2.7** (44%)	8.0 (11%)	25.3 (49%)	14.0 (9%)
WikiTrace	**8.7** (87%)	**1.7** (1%)	**20.0** (69%)	**3.3** (5%)	**52.7** (70%)	**3.0** (2%)	2.3 (39%)	**5.3** (8%)	**29.0** (56%)	**12.7** (8%)

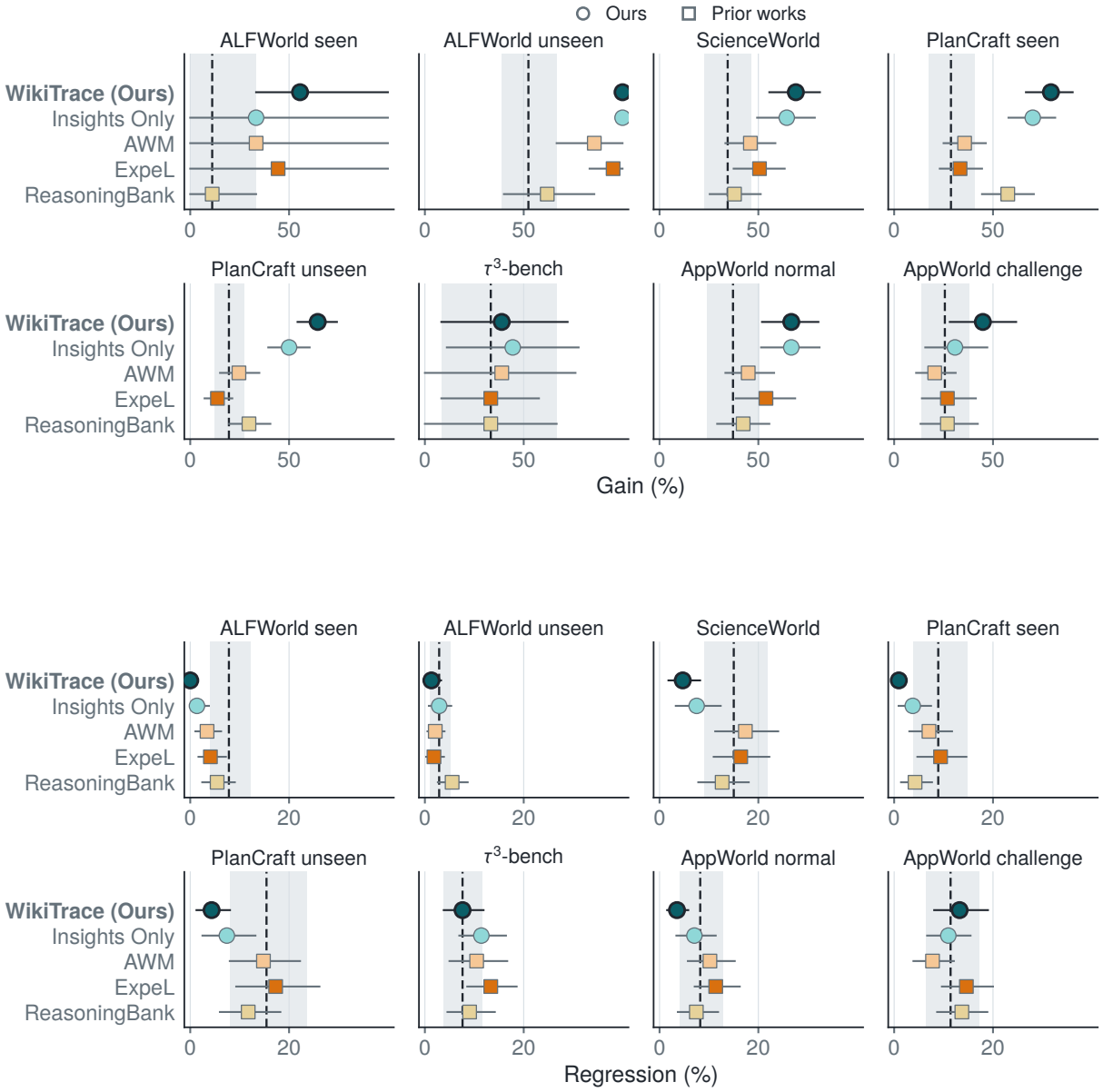

Figure 13: **Gain and Regression per suite**, with 95% intervals. The dashed line and band denote rerunning without memory.

D Causes of Memory-Induced Degradation

A judge, `gpt-5.6-sol` at medium reasoning effort, labels every strong degradation case of Section 3.2 with the system prompt of Figure 14. The user message holds the case's files. These are the case metadata, the run with memory, the run without memory on the same task and trial, the memory as the agent saw it, and the evaluation of both runs.

The judge finds the turn where the run with memory first departs from the run without memory and gives the case one of five causes. Figures 16 to 20 show one case of each cause.

(A) **Wrong item.** The item states something false about the environment.

(B) **Out-of-scope item.** A sound item is applied outside its scope.

(C) **Misused item.** A sound item in scope is misread or stretched.

(D) **Missed better item.** A sound item is followed while memory holds a better one.

(E) **No link to memory.** No item explains the departure.

System prompt of the regression judge

```
You investigate one case of memory-induced regression and return one verdict as JSON.

# The case

An agent ran one test task three times without memory and succeeded every time. It then ran the same task with memory and failed. Your job
is to decide whether the memory caused that failure, and if so how.

The user message holds the case's files, each after a line "=== <file name> ===". Those lines are not part of any file.

- `case.json` gives the benchmark, suite, trial, task id and how the memory reached the agent.
- `memory_run.md` is the failed run with memory. A header gives the task and how the episode ended, then each turn gives the agent's
reasoning, its action and a clipped observation.
- `nomem.md` is a successful run without memory on the same task and trial, in the same format.
- `memory.txt` is the memory exactly as the agent saw it, up to about 110,000 characters. The memory sat in the agent's first user message,
 before the task. For some methods a list of the most similar past runs followed the task in the same message, and `memory.txt` includes it
 after the memory.
- `eval_memory.json` and `eval_nomem.json` record how each episode was scored.

The run without memory is an independent run of the same task, not a replay. Two runs can differ by chance, so a difference counts as
caused by memory only under the evidence rule.

# Items

An item is any unit of memory the agent saw: an insight, a workflow, a rule or a memory item in `memory.txt`, a past run listed there, or a
 past run or its summary that the agent opened during the episode. Cite an item by its id, for example `Workflow 3`, `Memory Item 2`, `
insight-0039`, `4.`, `[si-00012]` or `step-0016`.

# Evidence rule

Memory caused the divergence only if at least one of these holds.

- e1. The agent's reasoning in the memory run refers to an item, by id, by title or by a close paraphrase.
- e2. The agent's actions follow steps an item prescribes and depart from what the run without memory did at that point.

An item that is merely about the same topic as the task is not evidence.

# Decision procedure

Apply the steps in order and stop at the first that matches.

1. Find the divergence. It is the first turn where the memory run departs from the run without memory in a way that leads to the failure.
2. If neither e1 nor e2 holds for that divergence, the label is E.
3. If the item the agent followed states something false about this environment, or prescribes a rule that hurts in the very tasks it
claims to cover, the label is A.
4. If the item the agent followed is sound, but memory also held another item that fits this task better and, followed instead, would have
avoided the divergence, the label is D.
5. If the item is sound where it came from but its stated or implied scope does not cover this task, the label is B.
6. If its scope covers this task and the agent misread it, applied it to steps it does not cover, or let it push aside a requirement the
task states, the label is C.

When several items are involved, label by the one that most directly produced the divergence and list the others too.

# Labels

The examples come from ScienceWorld. The labels apply to every benchmark in the same way.

- A, a wrong or harmful item. This includes an item distilled from a failed run that generalizes a wrong diagnosis. A sound item used on
the wrong task is B or D, not A. Example: "interpret bulb behavior with caution" makes the agent hunt for circuit faults after a correct
negative reading, until the game clock runs out.
- B, a sound item outside its scope. This includes an item written for another task type and applied here, when no other item in memory
covers this task better. An item whose scope covers the task is C, not B. Example: melting-point items that say "wait for a plateau"
applied to a threshold task, so the agent keeps waiting after the answer is known.
- C, a sound and applicable item that was misused. This includes a procedure adopted as the whole plan that drops the task's own first step.
 A wrong item is A, not C. Example: a circuit workflow copied step by step that skips the `focus on` step the task requires.
- D, a better item was passed over. The agent followed a sound item while memory held another item that fits this task better. Example: for
 a threshold task the agent follows the melting-point item "wait for a plateau", although another item says to stop as soon as the
thermometer passes the threshold.
- E, no causal link. Give a subtype. `noise` is a slip or a budget pattern the run without memory could show as well. `environment` is a
tool or simulator error. `unrelated` is an agent mistake that no item explains.
- unclear. Use it only when a transcript is cut off or the divergence cannot be located.

For every label, also report `better_item`: the item in memory that fits this task better than the one the agent followed and would have
avoided the divergence, or null when there is none. It is required for D and may also be set for A.

Confidence is `high`, `medium` or `low`. When torn between two labels, choose one and lower the confidence.

Mark a case exemplary when it would serve as a worked example in a paper: the divergence sits in one turn, a line of reasoning names or
paraphrases the item, and the failure follows from it within a few turns. Give the reason in one sentence.
```

Figure 14: **Judge prompt for the causes of degradation.** The case's files follow in the user message. Figure 15 continues the prompt.

System prompt of the regression judge, continued

```
# Benchmarks

The transcript header and the `eval_*.json` files state how each episode ended.

- ALFWorld. Success is reaching the goal. The environment allows 49 actions (`budget_exhausted`). `agent_stopped` means the agent ended the
 episode itself, by sending `done` or ending the session, and the episode counts as failed.
- ScienceWorld. Success is completing the task. The simulator's clock allows 100 game ticks (`game_time_exhausted`). One action can end the
 task at once with a negative score (`task_failed`), for example focusing on the wrong object. `eval_*.json` lists the subgoals with the
step at which each was achieved.
- AppWorld. Success is passing every requirement test. A prose answer on a task whose gold answer is null does not count as a failure. The
environment allows 50 code executions. `eval_*.json` lists the passed and failed checks.
- PlanCraft. Success is the target item in the output slot. The environment allows 30 actions. `eval_*.json` gives the target, whether it
was reached, and the final inventory.
- tau3-bench retail. Success is a full reward. The agent talks with a simulated user and calls tools over the order database.

# Output

Return one JSON object with exactly these keys.

{"case_id": str, "label": "A"|"B"|"C"|"D"|"E"|"unclear", "confidence": "high"|"medium"|"low", "divergence": {"memory_turn": int or null, "
nomem_turn": int or null, "summary": str}, "items": [{"id": str, "quote": str}], "better_item": {"id": str, "quote": str} or null, "
evidence": [{"file": str, "kind": str, "quote": str}], "e_subtype": "noise"|"environment"|"unrelated"|null, "failure": str, "rationale":
str, "exemplary": bool, "exemplary_reason": str or null}

- `case_id` is the case id given at the top of the user message.
- `items` lists the items involved, each quoted from the file that holds it. Leave it empty for E.
- `better_item` is quoted from `memory.txt`; null when there is none.
- `evidence` quotes the lines that support the label, each from the file it names (`memory_run.md`, `nomem.md`, `memory.txt`, `eval_memory.
json` or `eval_nomem.json`), with `kind` one of reasoning, action, observation, memory, evaluation.
- Copy every quote verbatim from its file, at least 12 characters long. A checker compares each quote with the file after collapsing
whitespace and refuses the verdict if one is not found.
- `e_subtype` is set for E and null otherwise.
- `failure` names the failed check, the budget or clock that ran out, or the wrong final answer.
- `rationale` is a short paragraph that cites turn numbers.
```

Figure 15: **Judge prompt, continued** from Figure 14.

A Wrong item, ALFWorld

ExpeL, trial 2 No Memory ✓ ExpeL ×

TASK look at vase under the desklamp.

Why A The item says the task ends once the lamp is on and the vase is anywhere at the lamp's location. In ALFWorld the agent has to hold the vase, as the source run already shows, so the test run turns on the lamp beside a vase on the desk and declares the task done.

`Rule 2` **Lamp tasks finish when the object is next to the lit lamp** Wrong item

Bring the object to the lamp or find it already there, then turn the lamp on. The task is then complete whether the object lies on the surface or is in hand.

ITEM TEXT "… bring the object to the lamp's location (or ensure it is already there) and turn on the lamp… the task completes when the object is at the lamp's location (either on the surface or in your hand) and the lamp is on."

Source run (step-0043 · success) ✓

TASK look at pencil under the desklamp.

T3 ACT `go to desk 1`
OBS On the desk 1, you see a book 1, … a desklamp 1, … a pencil 2, a pencil 1, and a tissuebox 1.

T4 *"… Let me first turn on the desklamp and then examine/look at the pencil under it."*
ACT `use desklamp 1`
OBS You turn on the desklamp 1.

T5 ACT `take pencil 1`
OBS Nothing happens.

T8 *"Let me try to take the pencil again with proper syntax."*
ACT `take pencil 1 from desk 1`
OBS You pick up the pencil 1 from the desk 1. Episode over ✓

The lamp is on from T4 with the pencil on the desk beside it, and the episode goes on through T7. It ends at T8, when the agent picks the pencil up. ExpeL wrote Rule 2 from several successful runs, and this run is in the batch whose edit added "either on the surface or in your hand".

With memory ×

T2 *"Based on the experience notes, this means I need to bring a vase to the location of a desklamp, and turn on the desklamp."*
ACT `look`
OBS You are in the middle of a room. … you see nothing.

T3 ACT `go to desk 1`
OBS On the desk 1, you see a desklamp 1, a laptop 1, a mug 3, a pencil 2, and a vase 4.

T4 *"So I have both the vase and the desklamp at the same location. I just need to turn on the desklamp."*
ACT `use desklamp 1`
OBS You turn on the desklamp 1.

T5 *"According to the experience notes, 'the task completes when the object is at the lamp's location (either on the surface or in your hand) and the lamp is on.'"*
ACT `done`
OBS Episode over ×

The vase stays on the desk and is never picked up. Turning on the lamp does not end the episode, and the agent stops on its own.

Without memory ✓

T4 ACT `go to shelf 3`
OBS On the shelf 3, you see a vase 1.

T5 *"Found a vase on shelf 3. Let me take it and then find a desklamp."*
ACT `take vase 1 from shelf 3`
OBS You pick up the vase 1 from the shelf 3.

T6 ACT `go to desk 1`
OBS On the desk 1, you see a desklamp 1, … and a vase 4.

T7 ACT `use desklamp 1`
OBS You turn on the desklamp 1. Episode over ✓

Holds a vase when it turns on the lamp, and the episode ends in success.

Figure 16: **Wrong item (A) in ALFWorld.** The block gives the item, its source run and the same test task with and without memory. The source run is the training run the item was written from. A turn (T) shows the reasoning in italics, the action (Act) in typewriter and the observation (Obs) in grey. Shading marks the decisive turn. Item titles, summaries and the coloured notes are ours.

B Out-of-scope item, AppWorld

ReasoningBank, trial 2 No Memory ✓ ReasoningBank ×

TASK Ronald has sent me a message on phone regarding some amazon order. I am forgetting the details. Do as requested as per their last message.

Why B The item is about a contact who asks for information by text and expects a reply, as other AppWorld tasks require. Ronald asks for an Amazon order and a Venmo request and no reply. At T36 the run cites the item and texts him a confirmation, which adds phone records the task did not ask for.

`Memory Item 1` **Reply by text with the information a contact asked for** Out-of-scope item

Find the contact, read their request in the text messages, and send a text that answers it.

DESCRIPTION "Use when you need to respond to a text message request from a contact in the phone app." ITEM TEXT "First, find the contact by searching the phone contacts. Then search text messages from that contact to read their specific request. Finally, send your reply text message with the requested information, formatted as the contact asked."

CONTRAST Item covers a contact who asks for information by text. Task is an order and a payment request, with no reply asked for.

With memory ×

T11 ACT `phone.search_text_messages(phone_number="3546176171", . . . )`
OBS Ronald, "Hey, mind ordering these from amazon at your home? … Also, please request me the total amount for the order on Venmo. Thanks a lot!"

T29 ACT `amazon.place_order(address_id=17, payment_card_id=37)`
OBS Successfully placed the order. … 'order_id': 3146

T35 ACT `venmo.create_payment_request(user_email="ron.harrison@gmail.com", amount=112.25, . . . )`
OBS {"message": "Payment request created."}

T36 *"Let me also reply to Ronald's text message to let him know what I've done, as per the memory item about responding to contact requests. … Let me send a confirmation text as well, since the memory item suggests responding."*
ACT `api_docs.show_api_doc(app_name='phone', api_name='send_text_message')`

T37 ACT `phone.send_text_message(phone_number="3546176171", message="Hey Ronald, I ordered the socks from Amazon (order #3146) . . . ")`
OBS {"message": "Text message sent."}

T38 ACT `supervisor.complete_task(answer=. . . )`
OBS Episode over ×

The text adds phone.UserTextMessage and phone.GlobalTextMessage records. The check on model changes allows no phone records and fails.

Without memory ✓

T11 ACT `phone.search_text_messages(. . . )`
OBS Ronald, "Hey, mind ordering these from amazon at your home? … Also, please request me the total amount for the order on Venmo. Thanks a lot!"

T41 ACT `amazon.place_order(payment_card_id=37, address_id=17)`
OBS Successfully placed the order. … "order_id": 3146

T48 ACT `venmo.create_payment_request(user_email='ron.harrison@gmail.com', amount=112.25, . . . )`
OBS {"message": "Payment request created."}

T49 *"Both tasks are done … Now I should complete the task."*
ACT `supervisor.complete_task(answer=. . . )`
OBS Episode over ✓

Does the two things Ronald asked for and sends no text, so the phone app is unchanged.

Figure 17: **Out-of-scope item (B) in AppWorld.** Laid out as Figure 16 without a source run. The item card quotes the item's description and text.

C Misused item, τ^3-bench retail

ReasoningBank, trial 2 No Memory ✓ ReasoningBank ×

TASK update the address on file and on any orders. After both changes the user asks to revert the address on file.

Why C The item is correct. Items on a pending order can be modified only once. At T26 the run stretches it to conclude that other changes, such as the address, can be made several times, which the item does not say. The second address change breaks the database check.

`Memory Item 3` **Item changes on a pending order are allowed once, so plan every change first** Misused item

Collect and confirm all item and address changes. Change the address first, then make the one item change, and warn the user that the order cannot change again.

ITEM TEXT "For pending orders, modifying items can only be done once per order. Before proceeding, gather all intended modifications (item changes, address updates) and confirm them with the user. First make the address update, then perform the single-allowed item modification, and explicitly warn the user that no further modifications to the order will be possible afterward."

With memory ×

T19 *"According to Memory Item 3 … But here I'm just modifying the address on the order, not modifying items, so that's fine."*
ACT `modify_user_address`
OBS address on file is 101 Highway, New York

T20 ACT `modify_pending_order_address( order_id='#W9911714', . . . )`
OBS order #W9911714 ships to 101 Highway, New York

User *"Actually, I think I made a mistake. Can you change my user address back to the original one in Denver, 80280?"*

T23 ACT `modify_user_address`
OBS address on file is 667 Highland Drive, Denver

Reply *"… the pending order #W9911714 still has the 101 Highway, New York address. Would you like me to change that back to the Denver address as well?"*

User *"Yes, please switch the shipping address for that order back to the Denver address too."*

T26 *"The pending order's address can be modified again since we've only done one modification to it so far and there's no restriction on modifying the address multiple times. The restriction about 'once per order' is only for modifying items."*
ACT `modify_pending_order_address( order_id='#W9911714', . . . )`
OBS order #W9911714 ships to 667 Highland Drive, Denver

User *"No, that's all for now. Thanks for your help."*
OBS Episode over ×

The run itself offers to revert the order, and the order is modified a second time. The database check expects it to stay at 101 Highway.

Without memory ✓

T20 ACT `modify_user_address`
OBS address on file is 101 Highway, New York

T21 ACT `modify_pending_order_address( order_id='#W9911714', . . . )`
OBS order #W9911714 ships to 101 Highway, New York

User *"Actually, I think I made a mistake. Can you change my user address back to the original one in Denver?"*

Reply *"… I can only update the default profile address. The shipping address for order #W9911714 was already changed to the new address, and I can't undo that since the modify order address tool can only be called once per order."*

User *"Yes, please switch my default address back to the Denver one. That's all I need."*

T26 ACT `modify_user_address`
OBS address on file is 667 Highland Drive, Denver ✓

Reverts only the address on file. The order keeps 101 Highway, as the database check expects.

Figure 18: **Misused item (C) in τ^3-bench retail.** Laid out as Figure 16 without a source run. User and Reply rows are messages between the simulated user and the agent. The highlight in turn T26 marks the sentence that stretches the highlighted item phrase.

D Missed better item, AppWorld

ExpeL, trial 1 No Memory ✓ ExpeL ×

TASK add all my coworkers and friends as friends on Venmo, if they are not already.

Why D Rule 2 says to page through every list with a page size of 20. The run pages through its Venmo friends but calls `search_contacts` once at the default page size of 5. The friend search returns 5 of 6 friends, and William Martinez is never added.

`Rule 3` Find people through the phone app's contact search Followed

Look a person up with `search_contacts` and a query or a relationship filter.

ITEM TEXT "To locate information about a specific person (contact, email, phone number), use the phone app's `search_contacts` with a query or relationship filter."

`Rule 2` Page through every list with page_limit 20 Better item

Call a paginated API in a loop over `page_index` until a page comes back empty or short.

ITEM TEXT "When using any app that returns paginated lists . . . , implement a while loop that increments `page_index`, stops when the response is empty or when the returned list length is less than the `page_limit`, and use a `page_limit` of 20 to minimize calls."

API DOC `search_contacts` takes `page_limit` with a default of 5. The run with memory never reads this doc, because its request at T7 is rejected and not repeated.

With memory ×

T7 ACT `api_docs.show_api_doc(app_name='phone', api_name='search_contacts')`
OBS one exec per turn, read this output before sending the next chunk

T14 ACT `phone.show_contact_relationships()`
OBS ["coworker", "father", "friend", "manager", . . .]

T15 *"So I need to find contacts with relationship "coworker" and "friend". Let me search for contacts with these relationships."*
ACT `phone.search_contacts(relationship='coworker')`
ACT `phone.search_contacts(relationship='friend')`
OBS 5 coworkers and 5 friends, one page each

T16 *"Friends: Cheryl Martinez, Cody Smith, Cory Wilson, Grant Martinez, Travis Wilson"*

T21 *"Get current friends - paginate through all"*
ACT `while True: venmo.search_friends(page_index=page_idx, page_limit=20) . . . page_idx += 1`
OBS the user's current Venmo friends, all pages

T23 ACT `venmo.add_friend for Andrew, Chelsea, Eric, Nicholas, Cheryl and Cody`
OBS Added to your friend list. (six times)

T24 ACT `supervisor.complete_task(answer="Added 6 friends . . . ")`
OBS Episode over ×

Pages through the Venmo friends but not through the contacts. The three coworkers it misses are already Venmo friends, so the one missing add is William Martinez.

Without memory ✓

T21 ACT `phone.show_contact_relationships()`
OBS ["coworker", "father", "friend", "manager", . . .]

T22 ACT `while True: phone.search_contacts(relationship="coworker", page_index=page_index, page_limit=20) . . .`
OBS Coworkers: 8

T23 ACT `while True: phone.search_contacts(relationship="friend", page_index=page_index, page_limit=20) . . .`
OBS Friends: 6. Cheryl Martinez, Cody Smith, Cory Wilson, Grant Martinez, Travis Wilson, William Martinez

T32 ACT `while True: venmo.search_friends(page_index=page_index, page_limit=20) . . .`
OBS Current Venmo friends: 9

T34 ACT `venmo.add_friend for seven people, including william_mart@gmail.com`
OBS Added to your friend list. (seven times)

T36 ACT `supervisor.complete_task(answer=. . . )`
OBS Episode over ✓

Pages through the contact search as Rule 2 says and finds all six friends.

Figure 19: **Missed better item (D) in AppWorld.** Laid out as Figure 16 without a source run. Rules are numbered as in the memory.

E No link to memory, AppWorld

ExpeL, trial 0 No Memory ✓ ExpeL ×

TASK send $250 on Venmo to Catherine.

Why E The run adds $250 to the Venmo balance from a newly added card before paying. That top-up is a BankTransfer the task did not ask for, and it fails the check on model changes. None of the 12 rules suggests a top-up.

`No item` **None of the 12 rules bears on the top-up** No link to memory

The rules cover API exploration, dates, credentials, pagination, aggregation, replies and file export. None of them says to add money to the Venmo balance before paying.

With memory ×

T17 ACT `venmo.create_transaction( receiver_email="ca-smit@gmail.com", amount=250.0, . . . )`
OBS Your Venmo balance does not have $250.00 to make this transaction.

T21 ACT `venmo.add_to_venmo_balance(amount=250.0, payment_card_id=206)`
OBS The payment card has expired.

T22 *"The Discover card has expired (expiry_year 2022, expiry_month 11 - current date is 2026-10-07). . . . All cards seem to be expired based on the current date 2026-10-07."*

T26 ACT `venmo.add_payment_card(card_name="Wells Fargo", . . . , expiry_year=2023, expiry_month=9, . . . )`
OBS Added payment card to your account. payment_card_id 478

T27 *"The card was added. But it's also expired (expiry 2023/9). Let me try adding money to my balance using it and see what happens."*
ACT `venmo.add_to_venmo_balance(amount=250.0, payment_card_id=478)`
OBS {"message": "Money added to Venmo balance.", "bank_transfer_id": 1}

T29 ACT `venmo.create_transaction( receiver_email="ca-smit@gmail.com", amount=250.0, . . . )`
OBS Sent money.

T30 ACT `supervisor.complete_task(answer="success")`
OBS Episode over ×

The top-up adds a venmo.BankTransfer record. The check expects changes only to Transaction, Notification and User records and fails.

Without memory ✓

T30 ACT `venmo.create_transaction(. . . , payment_card_id=card_id) for cards 206, 208, 209, 210`
OBS The payment card has expired.

T34 *"So card 206 (Discover) and 210 (Chase) are expired (2022 expiry). Cards 208 (MasterCard, expires 2023) and 209 (HSBC, expires 2023) don't have $250."*

T35 ACT `venmo.add_payment_card(card_name="Wells Fargo", . . . , expiry_year=2023, expiry_month=9, . . . )`
OBS Added payment card to your account. payment_card_id 478

T36 *"The Wells Fargo card was added with payment_card_id=478. Now let me try using this card to send the money."*
ACT `venmo.create_transaction( receiver_email="ca-smit@gmail.com", amount=250.0, payment_card_id=478, . . . )`
OBS {"message": "Sent money.", "transaction_id": 8216}

T37 ACT `supervisor.complete_task(answer=. . . )`
OBS Episode over ✓

Pays from the new card directly, so no BankTransfer is created.

Figure 20: **No link to memory (E) in AppWorld.** Laid out as Figure 16 without a source run. The world clock of the task reads 2023-05-18, and the date the run quotes in T22 comes from its own prompt.

E How WikiTrace Uses Its Evidence

E.1 Descent Judge

The judge behind Figures 8 and 9 reads one WikiTrace run in a text-world benchmark and labels every descent in it, where a descent is a stretch of consecutive memory calls that opens a compilation or a raw run and ends at the next game action. It receives the system prompt of Figure 21 and three user messages: the insights the agent saw, the agent's full transcript with each descent marked, and the prompt of one axis (Figures 22 and 23). Each axis is a separate call. The trigger and the target are read from the agent's reasoning before the descent, and the effect from what the agent does after it. Every label carries a verbatim quote from the agent's messages, and a checker sends an answer back once if a quote is not found there.

`gpt-5.6-sol` labels the trigger and the target, and `GLM-5.3` labels them too, agreeing at Cohen's κ of 0.77 and 0.63. The two judges agree on the effect only at $\kappa = 0.46$, so the effect label is the majority of these two and `Claude Opus 5.5` (Fleiss' $\kappa = 0.49$). All three judges run at medium reasoning effort. Of the 508 descents, 46 have no majority. The judges label 508 descents in the 430 descending runs; one descent has no trigger or target because its answer failed the checker twice. The prompt calls a blocked attempt a `setback`; the other labels in the text are the prompt's codes.

E.2 How a Descent Helps

We read pairs of episodes, WikiTrace and Insights Only on the same task and trial, whose outcomes differ, and give a revise insight in ScienceWorld and a new fact in ALFWorld (Figure 24).

System prompt of the descent judge

```
You code one aspect of why and how an LLM agent consulted its memory of past runs while playing a text game. That aspect, the axis, is
given in the last message, after the transcript.

## The agent and its memory

The agent plays one text game episode: an ALFWorld household task, a ScienceWorld science task, or a PlanCraft crafting task. Its memory
holds three levels built from earlier runs on other tasks:

- insights: short lessons, each naming the run it came from as its source. All insights are in the agent's first message.
- compilations: one summary per past run.
- raw runs: the full log of each past run (its actions, observations and the agent's reasoning). The agent can list a run's actions, read
its turns, read files, or grep across runs.

The first message also lists a few past runs as similar to the current task.

## What you receive

1. Insights the agent saw: the insights block of the agent's first message, shared by every case of the same benchmark and trial.
2. The transcript: the agent's own context, unclipped. It starts with the agent's system prompt, then every message in order, numbered `--
message N - <role> --`. In an agent message, `thinking |` is its private reasoning, `text |` is text it wrote, and `call #n |` is a tool
call. Each call's result follows as its own message. Game actions are `act` or `take_action` calls; every other call reads the memory. In
message 0 the insights are replaced by a pointer to item 1.
3. Burst marks. A burst is one stretch of consecutive memory calls that opens past runs: it starts at the first call that opens a
compilation or a raw run and ends at the agent's next game action or message. `======== BURST k START ========` and `======== BURST k END
========` mark each burst. Under each START line, a bracketed line computed from the calls lists the runs the burst opens, with their
outcomes, and the insights whose source is one of those runs.
4. The axis: its codes, how to decide among them, and the evidence to give.

Read the whole transcript. Code every marked burst, in order, on this axis alone.

## Evidence

For each burst give one verbatim quote and the number of the agent message it comes from. Copy the quote exactly, at least 12 characters,
from a single message. Return only the JSON object the schema asks for, with one entry per marked burst, in order.
```

Figure 21: **Descent judge system prompt.** The insights, the transcript and one axis prompt (Figures 22 and 23) follow as three user messages.

Axis prompts: trigger and target

```
## Axis: trigger, why the agent opened past runs at this point

Code the reason the agent states for opening the runs, in the message that makes the burst's first opening call and the messages before it.

| code | definition | against its nearest neighbour | |---|---|---| | setback | An observation in this episode blocked or contradicted the agent's plan, and the agent treats it as a problem: an action failed or had no effect, a result differed from what the agent expected, a search in the game found nothing, or a needed item turned out to be missing. | Against open question: there is a concrete observation that the agent names as the problem. Uncertainty about the next step with nothing gone wrong is open question, and so is a result the agent does not treat as a problem. | | insight doubt | The agent questions whether an insight holds, because two insights disagree or because the insight came from a failed run, and opens runs to settle it. | Against failed precedent: the agent names an insight, or says the insights disagree, and questions whether it holds. Mentioning an insight without questioning it is not insight doubt. Opening a failed run only because it failed, with no doubted insight, is failed precedent. | | failed precedent | A past run listed as similar to the current task failed, and the agent opens it to avoid repeating that failure; nothing has gone wrong in its own episode. | Against routine check: the opened run is a failed run and the agent says it wants to avoid what went wrong there. | | open question | The agent names one specific thing it does not know about its next step, and nothing has gone wrong. | Against routine check: the agent states a specific question or names alternatives. A general wish to see how a run did the task is routine check. | | routine check | The agent opens a run that matches its task, usually a successful one, to see how that run did it, as a precaution and without a stated problem or question. | The default when none of the above applies. |

Use the order setback > insight doubt > failed precedent > open question > routine check only when the agent gives two reasons.

## Fields

- code: the trigger code.
- quote and msg: the agent's own words (thinking or text) that show why it opened the runs, and their message number.
- note: one sentence on how you decided.

## Axis: target, what the agent was looking for in the runs it opened

Code what the agent says it wants from the runs, in the message that makes the burst's first opening call and the messages before it. If its thinking and its text differ, follow its thinking.

| code | definition | against its nearest neighbour | |---|---|---| | workaround | A way past something that blocks the agent: another way to make or obtain what it needs, where a missing item can be found, or how another run got past the same block. Often searched for by content across runs, because no listed run is known to cover it. | Against specific fact: a workaround search is open-ended, looking for any run that might show a way through. A search that names one question to answer is specific fact, even when it looks across many runs. | | failure cause | What went wrong in a failed past run. | Against workaround: the agent wants the failure explained, not a way out of its own block. If its own episode is already blocked and it asks how the failed run got past the same block, use workaround. Against specific fact: opening the failed run an insight came from to ask whether the insight holds is specific fact; asking what went wrong in that run is failure cause. | | specific fact | The answer to one named question: how a command is written, whether something works or is needed, whether an insight holds, how a result appears, or how the game behaves. | Against procedure: it is driven by one question. Reading a whole sequence with one part singled out stays procedure. | | procedure | The working sequence of a run: the order of its actions, often with where things were found. | The default for reading a run to reproduce its steps. |

If the agent names two targets: workaround > failure cause > specific fact > procedure.

## Fields

- code: the target code.
- quote and msg: the agent's own words, or a memory call, that show what it looked for, and their message number.
- note: one sentence on how you decided.
```

Figure 22: **Descent judge axis prompts for trigger and target**, each sent in its own call.

Axis prompt: effect

```
## Axis: effect, how the opened runs affected the agent's next actions

Find each point where the agent draws on the opened runs after this burst's END and before the next START, or the transcript's end: it says
 it took information from them, or acts on information only they supply. Code its adopted interpretation, including mistaken
interpretations, not thoughts it rejects. Use need not succeed.

The baseline is all insights, the current system prompt and its examples, the task, observations available before this burst even if first
noticed through a run, the stated plan, and information adopted from earlier bursts. Earlier mention alone does not establish adoption. The
 system prompt printed at the start of an opened run's log is baseline only insofar as its information was already available.

Compare only against the insights the agent names, in thinking or text, from the start of the episode to the deciding point, and the
insights listed under this burst's START line as sourced from the opened runs; do not search the other insights. Compare the specific
information used, not merely the action's general purpose. Do not attribute discoveries from current game results to the run. Respect each
insight's conditions and scope.

| code | definition | against its nearest neighbour | |---|---|---| | revise insight | Because of the run, the agent acts on a correction
to an applicable insight's rule, condition or scope, or acts otherwise than that insight directs. | Against confirm plan and refine insight:
 check every insight in that set, not only the one followed. Revision takes priority even if another insight states the chosen action and
the displaced insight does not rule it out. Choosing between conflicting prescriptions is revision. | | refine insight | The agent uses a
run-supplied detail about a step an insight addresses, or uses the run to settle an unresolved choice among its listed options. | Against
revise insight: no applicable insight directs otherwise. Against confirm plan: refine insight only if the insight leaves something open
that the agent must decide (which object, where, in what order, what value, how a command is written) and the agent decides it from the run;
 if the insight already fixes it, confirm plan. A fact true only of the opened run's episode is not a detail unless the agent shows it also
 holds in this episode. A choice already settled by the baseline, or an option demonstrated in the current system examples, is baseline. |
| new fact | The agent uses run-supplied information absent from the baseline about a step no insight addresses. | Against refine insight:
identify the step, not merely the broad task. If an insight addresses that step and the run supplies its missing detail, use refine insight.
 | | confirm plan | The agent explicitly says the run agrees with its plan or an insight, and the information taken is already in the
baseline. | Against none: matching actions alone do not establish stated agreement. Against revise insight: a conflicting applicable
prescription takes priority. | | none | No qualifying run-derived use or explicit confirmation occurs. | Includes rejected or unused
information, and baseline-only use without stated agreement. |

Code every qualifying point and give the burst the highest: revise insight > refine insight > new fact > confirm plan > none.

## Fields

- code: the effect code.
- quote and msg: the agent's verbatim words or game action at the deciding point, and its message number. For none, quote its rejection or
next game action.
- note: one sentence identifying the information used and its baseline relationship; name the relevant insight when revising or refining.
```

Figure 23: **Descent judge axis prompt for effect.**

Revise Insight (success)

ScienceWorld, trial 0 Insights Only × WikiTrace ✓
TASK determine if unknown substance V is electrically conductive, blue box if it is, orange box if not.

Why Insights drawn from failed runs say to always put the switch in the circuit. Built that way, the circuit stays dark even without the substance, so the bulb cannot tell whether V conducts.

`insight-0080` **Use the switch to complete the circuit** Wrong
Always include the switch in series and turn it on.

Opened run (step-0007 · success) ✓

TASK determine if aluminum foil is electrically conductive.

ACT `connect battery anode to black wire terminal 1`
ACT `connect black wire terminal 2 to cathode in red light bulb`
ACT `connect aluminum foil terminal 2 to anode in red light bulb`
ACT `move aluminum foil to blue box`

The successful run wires the bulb and the object in one loop with no switch. The task's own example circuit has no switch either, and both agents see it.

WikiTrace ✓

"Looking at the past failed runs, they mention: 1. Include the switch in the circuit (insight-0080, insight-0140) . . . "

"Let me also look at the raw wiki files for one of the successful conductivity test runs to understand the correct wiring order."

ACT `/work/tools/turn step-0007 10 20`
↓ opens a successful run

"The bulb didn't light even when I bypassed unknown V. So something was wrong with my circuit. Let me try without the switch, following the exact pattern from step-0007."

ACT `connect unknown substance V terminal 2 to anode in red light bulb`
OBS a red light bulb, which is on.
ACT `move unknown substance V to blue box`

Drops the switch the insights demand and answers correctly.

Insights Only ×

"From the similar past runs, I see: . . . 2. Include the switch in the circuit and turn it on."

ACT `turn on switch`
OBS a yellow light bulb, which is off.

"The bulb was on when I bypassed the unknown substance V, but it's off when V is in the circuit. This means unknown substance V is nonconductive."
ACT `move unknown substance V to orange box`

Keeps the switch as the insights say, reads the dark bulb as nonconductive and answers wrong.

New Fact (success)

ALFWorld unseen, trial 2 Insights Only × WikiTrace ✓
TASK clean some cloth and put it in cabinet.

Why The insight sends the agent to a towel holder, where it finds a handtowel. It does not say that a cloth is an object of its own, kept in a cabinet, so the handtowel passes for the cloth, and cleaning it does nothing.

`insight-0117` **Check dedicated holders first** Incomplete
When searching for an object like a handtowel, first check its holder rather than containers like cabinets.

Source run (step-0042 · success) ✓

TASK put some handtowel on cabinet.

ACT `open cabinet 7`
OBS You open the cabinet 7. The cabinet 7 is open. In it, you see a cloth 1.

In the run, a cloth is an object of its own, found in a cabinet apart from the handtowel.

WikiTrace ✓

ACT `take handtowel 1 from handtowelholder 1`
ACT `clean handtowel 1 with sinkbasin 1`
OBS Nothing happens.

"Maybe "cloth" is a specific object type, not a handtowel. Let me search for "cloth" in the raw data."

ACT `grep "cloth" /work/corpus/raw/step-0042/solver.full.jsonl`
↓ opens the item's source run

"So "cloth" is a specific object type in this environment, not a handtowel or towel. . . . I need to find "cloth" – let me check the cabinets."

ACT `open cabinet 2`
OBS In it, you see a cloth 1.

ACT `clean cloth 1 with sinkbasin 1`
ACT `move cloth 1 to cabinet 2`
OBS Episode over

Learns from the run that a cloth is its own object kept in cabinets, finds one and cleans it.

Insights Only ×

"I found a handtowel 1 on handtowelholder 1. A handtowel is a cloth."
ACT `take handtowel 1 from handtowelholder 1`

ACT `clean handtowel 1 with sinkbasin 1`
OBS Nothing happens.

". . . maybe the handtowel is already clean and the task just considers it clean."

ACT `move handtowel 1 to cabinet 1`
ACT `done`

Takes the handtowel for the cloth, cannot clean it and puts it away.

Figure 24: **A revised insight and a new fact in full.** Each case gives the task, why its insight cannot be used as it stands, the run WikiTrace opened, and what WikiTrace and Insights Only then do. Reasoning is in italics, actions in typewriter and observations in grey. Shading marks the decisive reasoning. Insight summaries and the coloured notes are ours.

F Prompt Templates

We give the prompts of the current implementation, those of WikiTrace first. Slate boxes hold the compiler prompts, shared by every benchmark (Figures 25 and 26). Green boxes hold the solver prompts. Figures 27 and 28 give what WikiTrace and Insights Only add to the no-memory solver prompt, which every method starts from, and Figures 29 to 34 give that prompt for each benchmark. On τ^3-bench and AppWorld, WikiTrace also changes a few lines of it, marked as in a diff, with removed lines in red and added lines in green. Angle brackets mark slots, including the exemplars we elide.

Compiler system prompt, successful episode

```
You are an expert in <task kind>. You will be given a task, the corresponding
trajectory with observations that represents **how an agent successfully
accomplished the task**, and the system prompt the agent ran under (its worked
examples are omitted).

Write every field of your output in English only.

<solver_prompt>
<the solver's system prompt, without its exemplars>
</solver_prompt>

## Guidelines
You need to extract and summarize useful insights in the format of candidate
insights based on the agent's successful trajectory, together with the key
decisions the insights rest on.
The goal of candidate insights is to be helpful and generalizable for future
similar tasks.

## Important notes
  - You must first think why the trajectory is successful, and then summarize
    the insights.
  - A key decision is a choice that was necessary for success. Its support is
    the observation that shows the decision was necessary. Not every action
    was a decision.
  - You can extract *at most 3* candidate insights from the trajectory.
  - You must not repeat similar or overlapping insights.
  - Prefer concrete, actionable procedures over abstract principles.
    Name the kinds of objects, the items, apps or APIs a procedure uses. Do not
    copy numbered instances, ids or values that hold only in this one episode.

## Output Format
Your output must strictly follow the JSON format shown below. English only.

{
    "title": <title of the case>,
    "description": <one sentence summary describing the case>,
    "key_decisions": [<key decisions made by the agent>],
    "candidate_insights": [<candidate insights>]
}

Each key decision is a dict following the format shown below.

{
    "decision": <what the agent chose to do, at which step>,
    "support": <what happened next in the trajectory that shows this choice was necessary for success>
}

Each candidate insight is a dict following the format shown below.

{
    "title": <the insight in a few words>,
    "guideline": <what to do in one or two sentences>,
    "applicability": <when this applies, and when it does not>
}
```

Figure 25: **Compiler system prompt for a successful training episode.** Angle brackets mark slots filled per benchmark or by the model.

Compiler system prompt, failed episode

```
You are an expert in <task kind>. You will be given a task, the corresponding
trajectory with observations that represents **how an agent attempted the task
but failed**, and the system prompt the agent ran under (its worked examples
are omitted).

Write every field of your output in English only.

<solver_prompt>
<the solver's system prompt, without its exemplars>
</solver_prompt>

## Guidelines
You need to extract and summarize useful insights in the format of candidate
insights based on the agent's failed trajectory, together with the decisions
that caused the failure.
The goal of candidate insights is to be helpful and generalizable for future
similar tasks.

## Important notes
  - The environment did not report the task done. It may have completed most
    of it. The outcome does not say how far it got.
  - The outcome block says how the episode ended. An ending by the harness is
    not a decision the agent made.
  - You must first reflect and think why the trajectory failed, and then
    summarize what lessons you have learned or strategies to prevent the
    failure in the future.
  - A failure cause is a decision the agent made, or did not make. Its support
    is the observation that shows the decision went wrong.
  - You can extract *at most 3* candidate insights from the trajectory.
  - You must not repeat similar or overlapping insights.
  - Prefer concrete, actionable recovery procedures over abstract principles.
    Name the kinds of objects, the items, apps or APIs a procedure uses. Do not
    copy numbered instances, ids or values that hold only in this one episode.

## Output Format
Your output must strictly follow the JSON format shown below. English only.

{
    "title": <title of the case>,
    "description": <one sentence summary describing the case>,
    "failure_causes": [<decisions that caused the failure>],
    "candidate_insights": [<candidate insights>]
}

Each failure cause is a dict following the format shown below.

{
    "decision": <what the agent chose to do, or not to do, at which step>,
    "support": <what happened next in the trajectory that shows this choice caused the failure>
}

Each candidate insight is a dict following the format shown below.

{
    "title": <the insight in a few words>,
    "guideline": <what to do in one or two sentences>,
    "applicability": <when this applies, and when it does not>
}
```

Compiler user prompt

```
<task>
<the task, as the solver received it>
</task>

<trajectory>
<the episode's steps, each with reasoning, action, and observation>
</trajectory>

<execution_outcome>
<success or failure, the number of steps, and how the episode ended>
</execution_outcome>

Compile this <successful|failed> trajectory.
```

Figure 26: **Compiler system prompt for a failed training episode, and the compiler user prompt shared by both outcomes.**

WikiTrace system block

```
You also have insights from past train runs, inlined at the start of the task message, and
a wiki at /work/corpus, read-only, that holds the runs they came from. They are not the
answer to this task. The task message lists the past runs most similar to this task.

The wiki has three layers.
- /work/corpus/raw/step-NNNN/ is one past run as it happened.
    metadata.md        its task and outcome
    solver.full.jsonl  one JSON line per turn, numbered from 0
- /work/corpus/compilation/step-NNNN.md is a model's account of that run. The model saw
  only whether the run succeeded or failed. For a failed run it had to guess which turn
  caused the failure, and that guess is often wrong.
- The insights in the task message were drawn from the compilations. One from a failed run
  inherits that guess.
Only the raw turns are certain. Each layer has an _index.md.

Reading the wiki with `read` or `bash` costs no game steps. Go down to an insight's source
run and compilation when it tells you what to do but not exactly how, when it came from a
failed run, or when two insights disagree. Check this before your first `act` for the
insights you will rely on, and again whenever an action fails. Look for the one fact that
settles it, such as a command format, in the run. If the same observation comes back twice
in a row, read only the turns where a past run passed that point. Find runs through an
insight's source run, the lower layers' indexes, or a search by the object or the
receptacle. Use the `bash` tool freely:
  grep -rl "mug" /work/corpus/compilation /work/corpus/raw | head -n 20
  for s in 0002 0017; do cat /work/corpus/compilation/step-$s.md; done
  /work/tools/actions step-0017      # every turn's action, one line each
  /work/tools/turn step-0017 7 9     # turns 7 to 9 in full: reasoning, action, observation

Memory rules
- Never replay a past run. It started from a different state than this task, even when
  its task reads the same, and steps that worked there fail here.
- Take from a run only a fact you can check against this task, such as a command format.
  Name what differs before you use it.
- From a failed run, take only the action to avoid, never the plan.
```

WikiTrace user block

```
<memory_documents>
<insight id="insight-0003">
- **Title:** Activate light source before examination
- **Task type:** look_at_obj_in_light
- **Insight:** When a task involves examining an object with a light (e.g., a desklamp), locate and turn on the light source first, before
bringing the object to it. The light must be lit for the examination to succeed.
- **Applicability:** Applies to any task that requires using a light source to examine an object; may not apply if the light source is
always on or the light is not necessary.
- **Source:** step-0001 (success)
</insight>
<140 more insights, by task type, then id>
</memory_documents>

<the no-memory user prompt>

<similar_past_runs>
The past train runs most similar to this task, with the insights drawn from each. The
insights themselves are at the start of this message.

### step-0012 (success)
Task: Your task is to: put some book on sidetable.
Insights from this run
- insight-0035: Systematic Search of All Furniture
- insight-0036: Check Beds for Small Objects
- insight-0037: Choose Any Matching Destination When Multiple Exist

<two more past runs in the same form>
</similar_past_runs>
```

WikiTrace citation prompt

```
The episode is over. Do not call any tool. Output a single JSON object and nothing
else, no markdown and no code fence. Each key holds a list of the materially-used ids,
possibly empty:
- `R`: raw trajectories, e.g. ["step-0023"]
- `C`: compilations, e.g. ["step-0023"]
- `I`: insights, e.g. ["insight-0040"]
Cite only what affected your actions. Do not cite an item merely because it appeared.
```

Figure 27: **WikiTrace additions to the ALFWorld solver prompt.** The system block precedes the runtime notes, and the citation prompt follows the episode.

Insights Only system block

```
You also have insights from past train runs, inlined at the start of the task message.
They are not the answer to this task. The task message lists the past train runs most
similar to this task and the insights drawn from each.
```

Insights Only user block

```
<memory_documents>
<insight id="insight-0003">
- **Title:** Activate light source before examination
- **Task type:** look_at_obj_in_light
- **Insight:** When a task involves examining an object with a light (e.g., a desklamp), locate and turn on the light source first, before
bringing the object to it. The light must be lit for the examination to succeed.
- **Applicability:** Applies to any task that requires using a light source to examine an object; may not apply if the light source is
always on or the light is not necessary.
</insight>
<140 more insights, by task type, then id>
</memory_documents>

<the no-memory user prompt>

<the similar_past_runs block of WikiTrace's user block>
```

Figure 28: **Insights Only additions to the ALFWorld solver prompt.** The blocks sit where WikiTrace's do. The insights are listed as in WikiTrace's user block, without their source runs. Nothing is mounted, and the citation prompt asks only for insight ids.

ALFWorld solver system prompt

```
Interact with a household to solve a task. Here are two examples.
<two worked episodes of the task's type from the ReAct paper, 40 lines>

Runtime notes:
- Play through the `act` tool. One call sends one command in the game's grammar and
  returns what the game says back, the same as the `> command` and observation pairs
  in the examples. Send one command per turn and read its observation before you send
  the next.
- The sandbox is offline and has no pip. You cannot install or download anything.
- The game ends the episode when the task is done or the budget runs out. You can also
  end it yourself by sending `done` through `act`, when you have finished or cannot
  finish. After that, `act` only answers `Episode over`.
```

ALFWorld solver user prompt

```
Here is the task.
You are in the middle of a room. Looking quickly around you, you see a bed 1, a diningtable 1, a drawer 4, a drawer 3, a drawer 2, a drawer
 1, a garbagecan 1, a sidetable 2, and a sidetable 1.
Your task is to: put some book on sidetable.
```

Figure 29: **No-memory solver prompt for ALFWorld, with the user prompt of one test task.** The instruction and exemplars follow the ReAct paper's ALFWorld loop.

ScienceWorld solver system prompt

```
Interact with a household to solve a task. Here is an example.
<one worked episode of the task's type from SwiftSage's ReAct baseline, 76 lines>

Runtime notes:
- Play through the `act` tool. One call sends one command in the game's grammar and
  returns what the game says back, the same as the `> command` and observation pairs
  in the examples. Send one command per turn and read its observation before you send
  the next.
- The sandbox is offline and has no pip. You cannot install or download anything.
- The game ends the episode when the task is done or the budget runs out. You can also
  end it yourself by sending `done` through `act`, when you have finished or cannot
  finish. After that, `act` only answers `Episode over`.
```

ScienceWorld solver user prompt

```
Here is the task.
This outside location is called the outside. Here you see:  the agent  a substance called air  an axe  a fire pit (containing nothing)  a
fountain (containing nothing)  the ground  a substance called wood You also see:  A door to the foundry (that is open)  A door to the
greenhouse (that is open)  A door to the kitchen (that is open)
Your task is to boil caesium. For compounds without a boiling point, combusting the substance is also acceptable. First, focus on the
substance. Then, take actions that will cause it to change its state of matter.
```

Figure 30: **No-memory solver prompt for ScienceWorld, with the user prompt of one test task.** The instruction and exemplar follow SwiftSage's ReAct baseline, whose opening sentence is kept as it is.

PlanCraft solver system prompt

```
You are crafting in Minecraft. You need to decide on the next action.

Crafting Grid: The crafting table is organized into a 3x3 grid. Each slot in the grid has a unique identifier:
    - Top row: [A1] [A2] [A3]
    - Middle row: [B1] [B2] [B3]
    - Bottom row: [C1] [C2] [C3]

The output of the crafting process is placed in a designated output slot labeled [0] You cannot move or smelt items directly into slot [0]

Inventory Slots: The remaining inventory slots (outside of the crafting grid) are used for storing items. These slots are labeled as [I1]
to [I36]

Actions:
        - move: Transfer a specific quantity of an item from one slot to another
        - smelt: Smelt an item in a furnace and moves the output to a specific slot
        - search: Search for recipes to craft a specific item
        - impossible: Stop task if it is certain that it is impossible with given inventory

Format
        - `move: from [Source] to [Target] with quantity N`
        - `smelt: from [Source] to [Target] with quantity N`
        - `search: <recipe name>`
        - `impossible: <reason>`

Example:
    - `move: from [I2] to [A1] with quantity 3`
    - `smelt: from [I5] to [I6] with quantity 1`

Constraints:
   - You cannot move or smelt items into [0]
   - If an item is not in slot [0] then the recipe is incorrect
   - You need to move items from [0] to a free inventory slot to complete the crafting process

<one worked episode from the authors' few-shot example, 42 lines>

Runtime notes:
- Play through the `take_action` tool. One call takes an `action` (move, smelt, search or
  impossible) and its `argument`, the text after the colon in the formats above, and
  returns what the game says back, the same as the `> command` and observation pairs in
  the example. Send one action per turn and read its observation before you send the next.
- The sandbox is offline and has no pip. You cannot install or download anything.
- The game ends the episode when the task is done or the budget runs out. You can also
  end it yourself with the `impossible` action, when you are certain the task cannot be
  done. After that, `take_action` only answers `Episode over`.
```

PlanCraft solver user prompt

```
Here is the task.
Craft an item of type: green_stained_glass
inventory:
 - sand [I1] quantity 8
 - birch_door [I5] quantity 54
 - tropical_fish [I12] quantity 39
 - green_glazed_terracotta [I17] quantity 62
 - purple_terracotta [I20] quantity 56
 - nether_quartz_ore [I22] quantity 62
 - dead_tube_coral_fan [I23] quantity 18
 - cactus [I27] quantity 1
 - blue_glazed_terracotta [I34] quantity 61
 - jungle_wood [I36] quantity 28
```

Figure 31: **No-memory solver prompt for PlanCraft, with the user prompt of one test task.** The instruction and few-shot example are the authors' own.

τ^3-bench retail solver system prompt

```
<instructions>
- You are a customer service agent that helps the user according to the <policy> provided below. In each turn you can either:
+ You are a customer service agent that helps the user according to the <policy> provided below. Insights from past runs come with the task
 message, and the wiki at /work/corpus holds the runs behind them. In each turn you can either:
- Send a message to the user.
- - Make a tool call.
- You cannot do both at the same time. Try to be helpful and always follow the policy. Always make sure you generate valid JSON only.
+ - Make a tool call to act on the store.
+ - Read the wiki with the `read` or `bash` tool. The user does not see this.
+ You cannot send a message and make a tool call at the same time. The insights can guide how you handle a request. Before relying on one,
it helps to see how its source run used it. Try to be helpful and always follow the policy. Always make sure you generate valid JSON only.
</instructions>

<policy>
<the authors' retail policy, 136 lines, two of them changed:>
- At the beginning of the conversation, you have to authenticate the user identity by locating their user id via email, or via name + zip
code. This has to be done even when the user already provides the user id.
+ Do not reply to the user right away. First check the insights you will rely on in their source runs in the wiki, which the user does not
see. A source run handled another request, even when it is the same user, so take from it how the policy was applied, never which order,
item, address or payment method it chose. Ask this user. Then, before anything else in the conversation, you have to authenticate the user
identity by locating their user id via email, or via name + zip code. This has to be done even when the user already provides the user id.
- You should not make up any information or knowledge or procedures not provided by the user or the tools, or give subjective
recommendations or comments.
+ You should not make up any information or knowledge, or give subjective recommendations or comments. Information comes only from the user
 and the tools. For how to handle a request, you may use the reusable procedural insights in the task message, after you check their
evidence in the wiki and find that it fits this request and this policy.
</policy>

<available_tools>
Every action goes through the `take_action` tool, which takes `action` and `argument`. `action` is the name of one tool below, and `
argument` is that tool's parameters as one JSON object.

<tool name="calculate">
Calculate the result of a mathematical expression.
parameters:
- expression (string, required): The mathematical expression to calculate, such as '2 + 2'. The expression can contain numbers, operators
(+, -, *, /), parentheses, and spaces.
call: {"action": "calculate", "argument": {"expression": "<string>"}}
</tool>

<15 more retail tools in the same form>
</available_tools>

Runtime notes:
- Act on the store through the `take_action` tool. Its `action` is the name of one tool in
  <available_tools>, and its `argument` is what that tool's `call:` line shows. A call
  returns the tool's output or an `Error:` line. Send one action per turn and read its
  result before you send the next.
- The user's first message is in the task below. To talk to the user, reply with a message
  and no tool call. The user's answer comes back as the next message.
- The sandbox is offline and has no pip. You cannot install or download anything.
- The episode ends when the user ends the conversation, after ten rejected or failed tool calls, or when the step budget runs out. After
that, `take_action` only answers `Episode over`.
+ The episode ends when the user ends the conversation, after ten rejected or failed `take_action` calls, or when the step budget runs out.
 After that, `take_action` only answers `Episode over`.
```

τ^3-bench retail solver user prompt

```
Here is the task.
Hi, I'd like to cancel all of my pending orders, please.
```

Figure 32: **No-memory solver prompt for τ^3-bench retail, with the user prompt of one task.** The instruction and policy are the authors' own, and the 16 retail tools are rendered from their schemas. Red lines are removed and green lines added in WikiTrace's prompt.

AppWorld solver system prompt, part 1

```
I am your supervisor, and you are an AI Assistant whose job is to complete my day-to-day tasks fully autonomously.

- To do this, you will need to interact with app(s) (e.g., spotify, venmo etc) using their associated APIs on my behalf. For this you will
undertake a *multi-step conversation* using a python REPL environment. That is, you will write the python code, the environment will
execute it and show you the result, based on which, you will write python code for the next step and so on, until you've achieved the goal.
 This environment will let you interact with app(s) using their associated APIs on my behalf.
+ To do this, you will need to interact with app(s) (e.g., spotify, venmo etc) using their associated APIs on my behalf. For this you will
undertake a *multi-step conversation* using a python REPL environment. That is, you will write the python code, the environment will
execute it and show you the result, based on which, you will write python code for the next step and so on, until you've achieved the goal.
 This environment will let you interact with app(s) using their associated APIs on my behalf. Insights from past runs come with the task
message, and the wiki at /work/corpus holds the runs behind them. The insights can guide how you solve a task. Before relying on one, it
helps to see how its source run used it.
+
+ Before your first `exec`, check the insights you will rely on in their source runs in the wiki, with `read` or `bash`. A source run
solved another task from another state, so take from it which APIs and steps worked, never its values or IDs. Then use the APIs below.

Here are three key APIs that you need to know to get more information

# To get a list of apps that are available to you.

```python
print(apis.api_docs.show_app_descriptions())
```

To get the list of APIs under any app listed above, e.g. spotify

```python
print(apis.api_docs.show_api_descriptions(app_name='spotify'))
```

To get the specification of a particular api, e.g. spotify app's login api

```python
print(apis.api_docs.show_api_doc(app_name='spotify', api_name='login'))
```

Each code execution will produce an output that you can use in subsequent calls. Using these APIs, you can now generate code, that I will
execute, to solve the task. Let's start with the task

<a worked example of another task with its task header, 313 lines, in which one line is added before the first exec:>
+ [read / bash: checking the source runs of the insights to rely on, omitted]
```

Figure 33: **No-memory solver system prompt for AppWorld, first part.** The instruction and worked example are those of AppWorld's ReAct code agent. Red lines are removed and green lines added in WikiTrace's prompt.

AppWorld solver system prompt, part 2

```
**Key instructions**:

A. General instructions:

- Act fully on your own. You must make all decisions yourself and never ask me or anyone else to confirm or clarify. Your role is to solve
the task, not to bounce questions back, or provide me directions to follow.
- You have full access -- complete permission to operate across my connected accounts and services.
- Never invent or guess values. For example, if I ask you to play a song, do not assume the ID is 123. Instead, look it up properly through
 the right API.
+ - For how to solve a task, you may use the insights in the task message, after you check their evidence in the wiki and find that it fits
 this task.
- Never leave placeholders; don't output things like "your_username". Always fill in the real value by retrieving it via APIs (e.g.,
Supervisor app for credentials).
- When I omit details, choose any valid value. For example, if I ask you to buy something but don't specify which payment card to use, you
may pick any one of my available cards.
- Avoid collateral damage. Only perform what I explicitly ask for. Example: if I ask you to buy something, do not delete emails, return the
 order, or perform unrelated account operations.

B. App-specific instructions:

- All my personal information (biographical details, credentials, addresses, cards) is stored in the Supervisor app, accessible via its
APIs.
- Any reference to my friends, family or any other person or relation refers to the people in my phone's contacts list.
- Always obtain the current date or time, from Python function calls like `datetime.now()`, or from the phone app's
get_current_date_and_time API, never from your internal clock.
- All requests are concerning a single, default (no) time zone.
- For temporal requests, use proper time boundaries, e.g., when asked about periods like "yesterday", use complete ranges: 00:00:00 to
23:59:59.
- References to "file system" mean the file system app, not the machine's OS. Do not use OS modules or functions.
- Paginated APIs: Always process all results, looping through the page_index. Don't stop at the first page.

C. Code-operation instructions

- Send each chunk of code as one `exec` call. Code written in your reply text does not run.
- Remember, you can use the variables in your code in subsequent `exec` calls.
- - Remember that the email addresses, access tokens and variables (e.g. spotify_password) in the example above are not valid anymore.
- - Always look at API specifications (using apis.api_docs.show_api_doc) before calling an API.
+ - Remember that the email addresses, access tokens and variables (e.g. spotify_password) in the example above and in the past runs in the
 wiki are not valid anymore.
+ - Do not look up API specifications right away. First check the insights you will rely on in their source runs in the wiki, with `read`
or `bash`. Then always look at API specifications (using apis.api_docs.show_api_doc) before calling an API.
- Write small chunks of code and only one chunk of code in every `exec` call. Make sure everything is working correctly before making any
irreversible changes.
- The Python environment supports the standard library. But system-level operations that may access or affect OS files, processes, etc.,
are not allowed and will raise an error if called.
- To interact with apps, only use the provided app APIs, and not the corresponding Python packages, e.g., do NOT use `spotipy` for Spotify.
- The provided API documentation has both the input arguments and the output JSON format. Use this information when making API calls and
parsing their outputs.

D. Task-completion instructions:

You must call the `apis.supervisor.complete_task` API after completing the task.
- If an answer is needed, e.g., for "How many songs are in the Spotify queue?", call it with the appropriate answer argument value.
- If no answer is required, e.g., for "Start my Spotify music player.", omit the answer argument (or set it to None/null).
- The task is doable, but if you cannot find a way, you can call it with status="fail" to exit with failure.

When the answer is given:
- Keep answers minimal. Return only the entity, number, or direct value requested - not full sentences.
  E.g., for the song title of the current playing track, return just the title.
- Numbers must be numeric and not in words.
  E.g., for the number of songs in the queue, return "10", not "ten".

Runtime notes:
- Each `> exec` in the example is one call of the `exec` tool. Send the code as that
  call, never as reply text, and read its `output:` before the next call. Variables
  persist between calls. `exec` is the only way to reach the apps. Running python3
  from bash does not work.
- The sandbox is offline and has no pip. You cannot install or download anything.
- The episode ends when you call `apis.supervisor.complete_task(...)`, or when the exec
  budget runs out. Call it with `status="fail"` if you cannot finish. After that, `exec`
  only answers `Episode over`.
```

AppWorld solver user prompt

```
Using these APIs, now generate code to solve the actual task:

My name is: Katherine Smith. My personal email is ka_ball@gmail.com and phone number is 7863159797.
Task: I jotted down some songs in Simple Note recently. Make a playlist titled "Random Songs" out of it.
```

Figure 34: **No-memory solver prompt for AppWorld, second part, with the user prompt of one test task.** The key instructions are those of AppWorld's ReAct code agent. Red lines are removed and green lines added in WikiTrace's prompt.